| TradeSense WBA | Status | Date |
|---|---|---|
| V 10.0.0 | Released | 10/01/2023 |
| V 10.0.1 | Released | 10/02/2023 |
| V 10.0.2 | Released | 05/04/2023 |
| V 10.0.4 | Released | 18/07/2023 |
| V 10.0.5 | Released | 24/08/2023 |
| V 10.0.6 | Released | 02/10/2023 |
| V 10.0.7 | Released | 15/11/2023 |
| V 10.0.8 | Released | 23/01/2024 |
| V 10.0.12 | Released | 09/07/2024 |
| V 10.0.14 | | 30/10/2024 |

**TradeSense Websocket Backend API**
- Introduction

**Technical description**
- Message format
  - Datetime format
  - Command message
  - Response message
  - Notify message
  - Error handling
- User Creation
- Authentication
  - WBA Client Manager in Assure UI
  - Use Cases
    - Case 1 : Client API status
    - Case 2 : Client session is about to expire
    - Case 3 : Newly created token status
    - Case 4 : If client authenticated in past and is now expired or disconnected
- Re-authentication
  - Configuration
  - Use Cases
    - Case 1 : Settings Auth Timeout and Auth Gracetime are neither configured or set the value as 0
    - Case 2 : Client Re-auths within Gracetime
    - Case 3 : Client re-auths before Gracetime
    - Case 4 : Client fails to re-auth itself during Gracetime
    - Case 5 : Client re-authenticates with different token
    - Case 6 : Settings Auth Gracetime  is greater than  Auth Timeout
  - Authentication Responses
  - Errors
- Commands
  - Fetch data
    - Get Zones
      - Optional arguments
      - Errors
    - Get Turrets
      - Optional arguments
      - Errors
    - Get Users
      - Optional arguments
      - Errors
    - Get Events
      - Limitation
      - Using ID
      - Using datetime
      - Response
      - Errors
    - Get Calls
      - Limitation
      - Using ID
      - Using datetime
      - Using call_ref
      - Using filter
      - Response
      - Call States
      - Errors

- Get Version
  - Response
  - Errors
- Get Tpos
  - Optional arguments
  - Errors
- Get Lines
  - Optional arguments
  - Errors
- Get Shared Profiles
  - Optional arguments
  - Errors
- Get Health Api Report
  - Response
  - Errors
- Call a service
  - Click To Dial
    - Place a new outbound call
      - Limitation
      - Local extension availability
      - Outgoing call errors
      - Response
    - Accept an incoming call
    - Hold an established call
    - Resume a held call
    - Terminate an opened call
    - Dispatch an opened call
    - Transfer an opened call
      - Limitation
      - Attended
      - Unattended
    - Make a conference call
      - Limitation
    - Remotely Logon/Logoff a user on turret
      - Limitation
      - Logon
      - Logoff
    - Errors
- Subscribe to Notifications
  - Errors
- Unsubscribe to Notifications

Notifications
- Calls notifications
  - Outgoing call scenario
  - Error handling
- Presence notifications
  - Presence state values
- Alerts notifications

Settings
- Ping settings
  - Enable the ping mechanism
  - Configure the ping interval
- Notifications settings
  - Configure the alerts notification on WBA
- Get Response batch size settings
  - Response batch settings

Get Response limit settings
Response limit settings
Time out settings
Configure the time out interval for click to dial requests
Configure the polling rate interval for click to dial monitoring
Configure the time out interval for requests
Outdated data settings
Configure the fetch interval for WBA requests
Support
Requirements
Startup
Provisioning
Cluster Settings Update
Errors
Commands
Log Level
Get the current log level
Change the log level
Available levels
Errors
Get Health
Get the provisioning status
Provisioning data items
Errors
Document History

# TradeSense Websocket Backend API

> Released from 9.5.0, TradeSense Websocket Backend API (WBA) becomes the standard for using the Click To Dial (CTD) feature.
> The old CTD API remains supported but will not be maintained.

## Introduction

This document gives an overview about the functionalities that can be implemented by any third parties that wish to leverage TradeSense data in their bigger pipe of data that implements WebSockets.

The WBA allows to:

- post Click To Dial (CTD) requests;
- get notification for all the Customer Relationship Management (CRM) events.

# Technical description

## Message format

The API uses JSON serialized objects.

### Datetime format

Datetime information are formatted as UTC datetime string following the ISO 8601 representation.

> Example: "2018-07-06T12:13:17Z"

### Command message

This object always contains the following keys:

- `command` : type of the command;
- `command_ref` : the reference of the command is a string value generated by the client-side. This reference is used to identify the response linked to client's request;
- `args` : command arguments (optional);

```
{
   "command":"command type",
   "command_ref":"command reference",
   "args":{}
}
```

## Response message

The server-side will respond to each command with a message indicating if the command is done and if it was successful. It will contain a `data` key containing the data object.

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{}
}
```

## Notify message

Client-side is able to subscribe to different types of notification. When subscribed, server-side will send a message every second with all last notifications not yet received by the client.

The notification object sent by the server always contains following keys:

- `command` : the command type of a notification message will always be `notify`
- `data` : object sent (refer to Notification section for more information)

```
{
   "command":"notify",
   "data":{
      "..."
   }
}
```

## Error handling

If an error occurs, the server will send a response message with `success` set to `false`. The `error` key contains an object with two keys: `code` and `message`.

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":false,
   "error":{
      "code":"code",
      "status":"status",
      "message":"message",
      "reason":"message for user"
   }
}
```

| Code | Status | Message |
|---|---|---|
| **400** | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values |
| **401** | Unauthorized | You lack permissions to access this resource |
| **404** | Not Found | Resource Not Found |
| **500** | Server Error | Internal Server Error |
| **498** | Invalid Token | The client had not sent previously used token. If client needs new session, please disconnect and connect with new token |
| **409** | Conflict | User already logged on |
| **412** | Bad Request | Pre Condition Failed |

In addition, a reason can be provided to help the user understand what went wrong.

Bad request reasons can be :

- Unable to deserialize message
- Mandatory field/parameter missing
- Command not found
- Action not found
- Args missing for ClicktoDial command
- Missing mandatory parameter: `parameter_name`
- Parameter `parameter_name` is not a valid `expected_type`

If the configured time out interval for requests is reached (see settings section)

- Request timed out

# User Creation

You will need a valid token to connect your WebSocket implementation. This token is generated by creating a new API user on Assure.

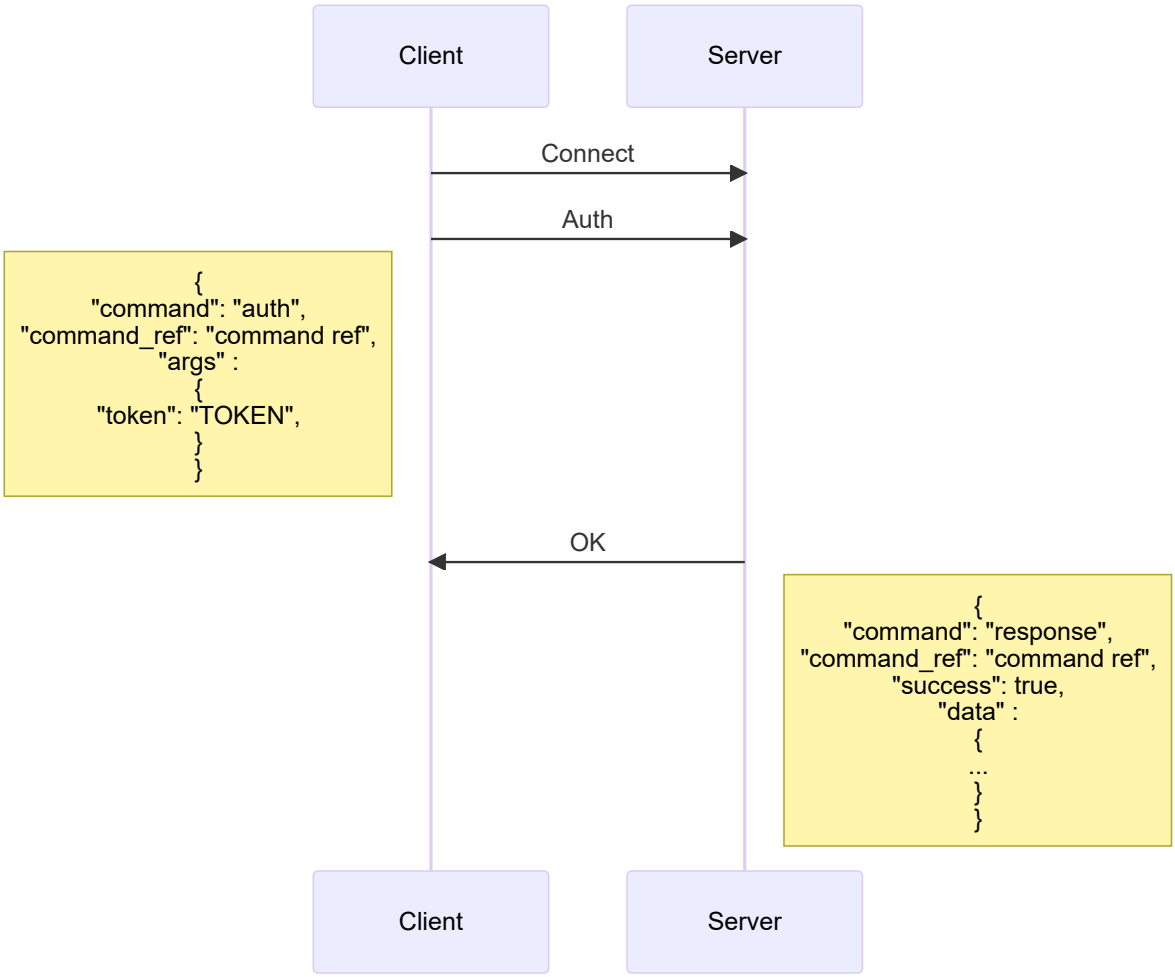
> https://TradeSenseFQDN/assure/admin/wba/new

Connect your implementation to:

> wss://TradeSenseFQDN/api

Use the token to authenticate your client-side application

# Authentication




Authentication via API token is mandatory. When the client-side code decides to open a WebSocket, it contacts the server-side using a command `auth` with the `token` as argument :

```
{
   "command":"auth",
   "command_ref":"command reference",
   "args":{
      "token":"TOKEN"
   }
}
```

If the client-side supplies valid authentication, the authentication phase will complete by the server sending the response message:

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{}
}
```

# WBA Client Manager in Assure UI

A new WBA API UI has been added to Administrator menu in Assure as shown below,

From the "WBA API" UI , Admin can create/delete new API users. This new UI also displays all information like Token, Name, API status, Client Last Authenticated time, Last Refresh time and Actions allowed on the API session. The Actions column consists of two buttons Force Close and Re-Authenticate. Both Actions buttons are enabled only when API Client Status is in 'Authenticated'.

- `Force Close` triggers the server to close the web socket connection of the connected client and the token will be released.
- `Re-Authenticate` triggers a re-authenticate notification to the connected client and this triggers the connected client to re-authenticate itself.
- `Last Refresh` time is updated every 10 secs after the client is authenticated and until the client is disconnected.

**API Client Status**

| API Status | Description |
|---|---|
| Authenticated | Client is successfully Authenticated with this token |
| Expire in x secs | Client Session Expiry information |
| Not in Use | No Client Session exist for this token |

# Use Cases

## Case 1 : Client API status

When a Client connects and successfully authenticates with WBA, the API status, Client last authenticated time, and actions are updated in the WBA Clients Management Screen.

## Case 2 : Client session is about to expire

If the Client Session is about to expire as per the settings in TSS:

- application.global.wba.auth.timeout = X seconds
- application.global.wba.auth.gracetime = Y seconds
  When the Client's session is timed out after X seconds, API status is updated as 'Expire in Y secs' until the grace time is exceeded in WBA Clients Management Screen.

## Case 3 : Newly created token status

If a token is just created and client is not authenticated, then the fields (Last Authenticated time, and actions) are not updated with any data. The API Status is updated as 'Not in Use' by default in WBA Clients Management Screen.

### Case 4 : If client authenticated in past and is now expired or disconnected

As soon as a client gets disconnected or a client session is expired, the Client data will be deleted , so the API Status  will be displayed as 'Not in Use' for that token in WBA Clients Management Screen.

# Re-authentication

To avoid having long connected client session, TradeSense has introduced a session refresh mechanism. This means clients needs to reauthenticate their session after a specific time. This session authentication time is configurable through TSS. The client shall be notified by server before the expiry time to re-authenticate itself.

## Configuration

The following two configurable settings has been introduced. It can be configured at TSS -> Productivity tools clusters -> Select the Cluster -> Settings. After these configuration the assure stack needs to be restarted.

| Name | Settings | Description | Default value (seconds) |
|---|---|---|---|
| Auth Timeout | application.global.wba.auth.timeout | This is time duration after successful authentication or re-authentication, the client session will expire and will be disconnected by the server, if not reauthenticated. | 0 |
| Auth Grace Time | application.global.wba.auth.gracetime | This is a time in seconds before the authentication timeout, a server notification will be sent to client to re-authenticate itself with the same token. Client needs to reauthenticate only after the receipt of this notification | 60 |

Note : By default(value of 0), reauthentication mechanism will be disabled, so it will be backward compatible for existing client implementations. But it is highly recommended for clients to implement this mechanism in their client applications.

The value of Timeout should always be greater than Gracetime.

For example,
application.global.wba.auth.timeout = 1800
application.global.wba.auth.gracetime = 60

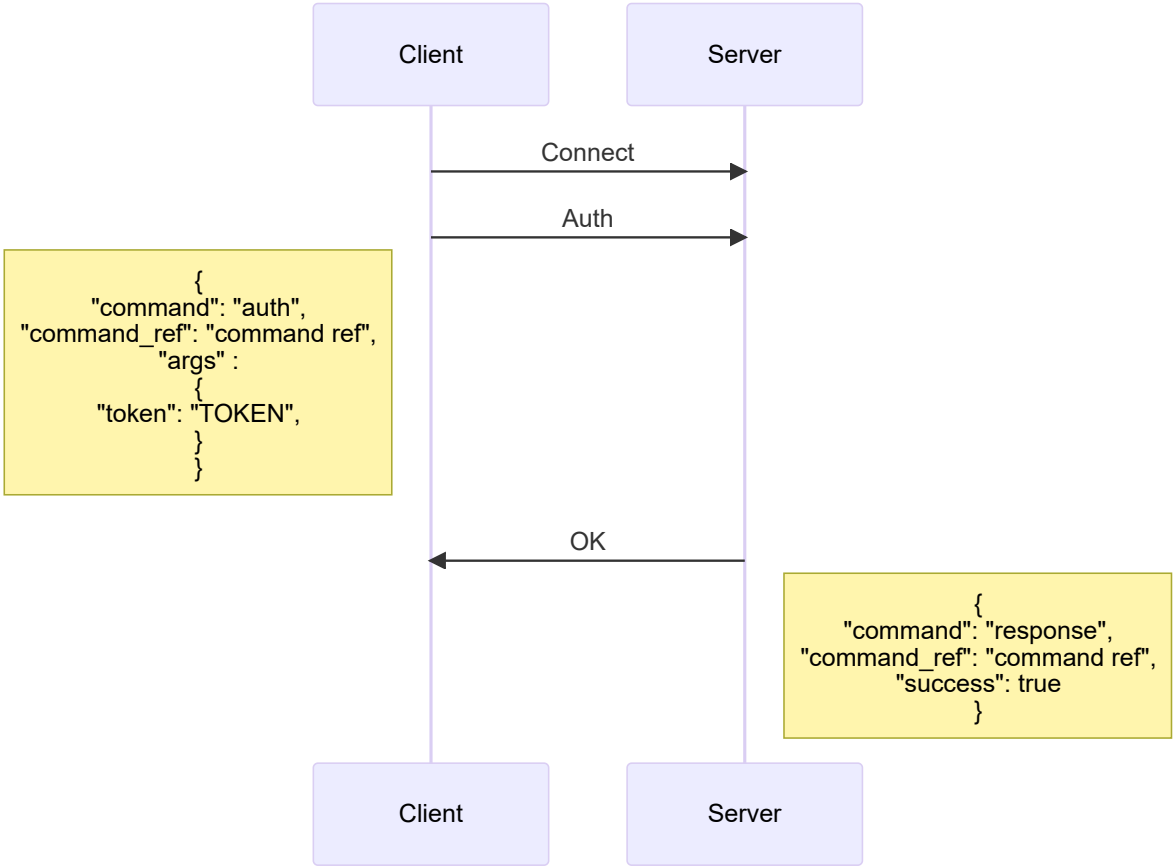
## Use Cases

Different re-authentication use cases are described below,

### Case 1 : Settings Auth Timeout and Auth Gracetime are neither configured or set the value as 0

application.global.wba.auth.timeout = 0
or
application.global.wba.auth.gracetime = 0

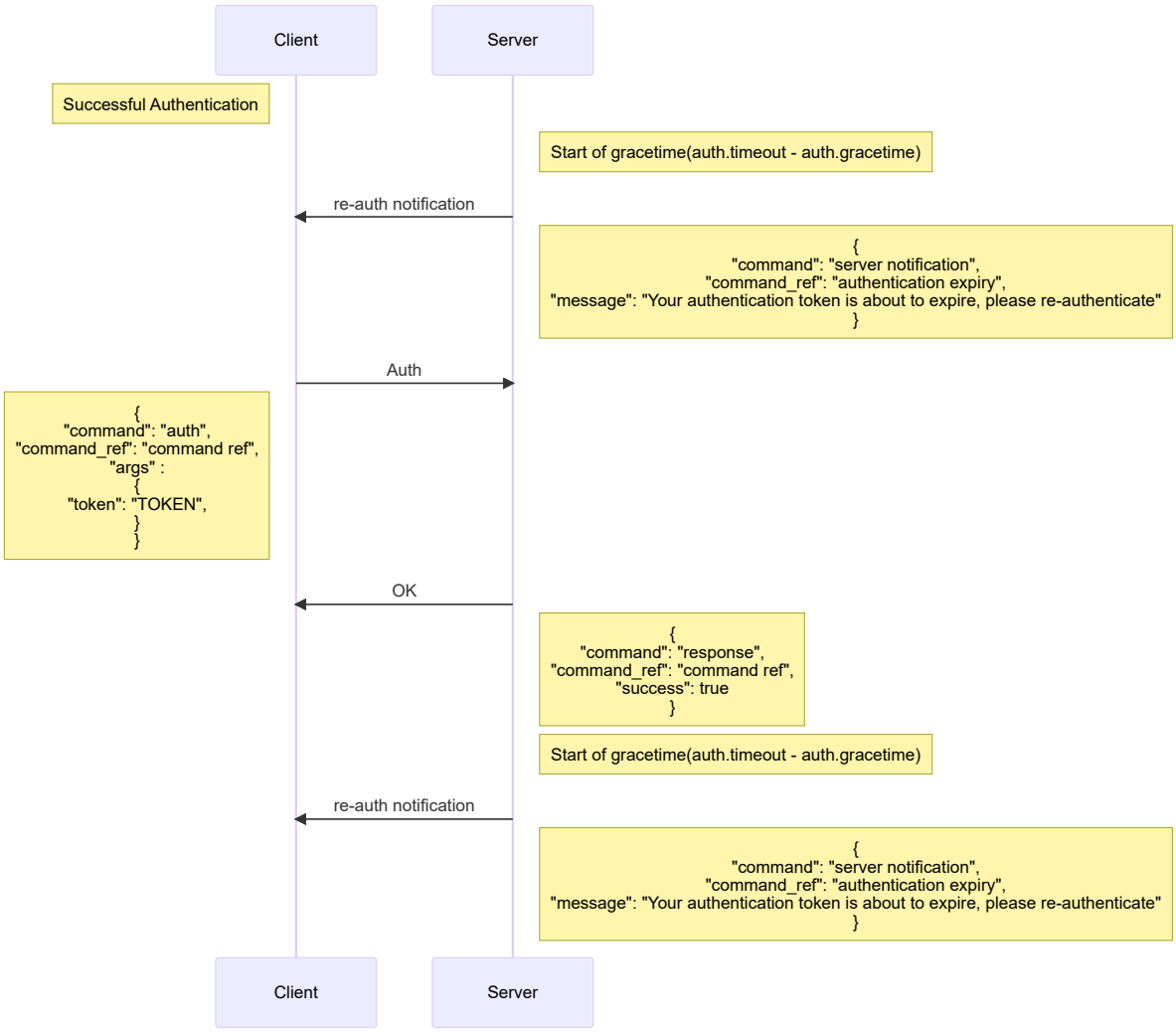
The re-authentication process is disabled, client authentication cycle is as same current behaviour. Once authenticated, the session is alive till the client disconnects.




### Case 2 : Client Re-auths within Gracetime

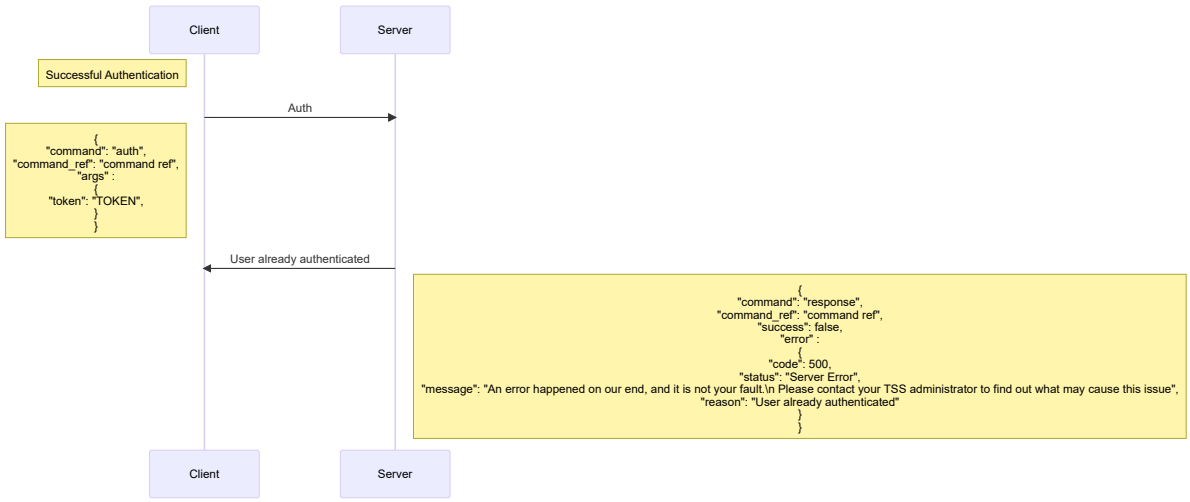
application.global.wba.auth.timeout = 60
application.global.wba.auth.gracetime = 30
Grace Time(60-30) = 30

The server will send re-auth notification at the start of the Auth Gracetime. If client re-authenticates  within grace period (30 seconds) after receiving server re-authentication notification, client session is refreshed. Server will repeat this re-auth mechanism until Client/Server websocket disconnects or error cases.




## Case 3 : Client re-auths before Gracetime

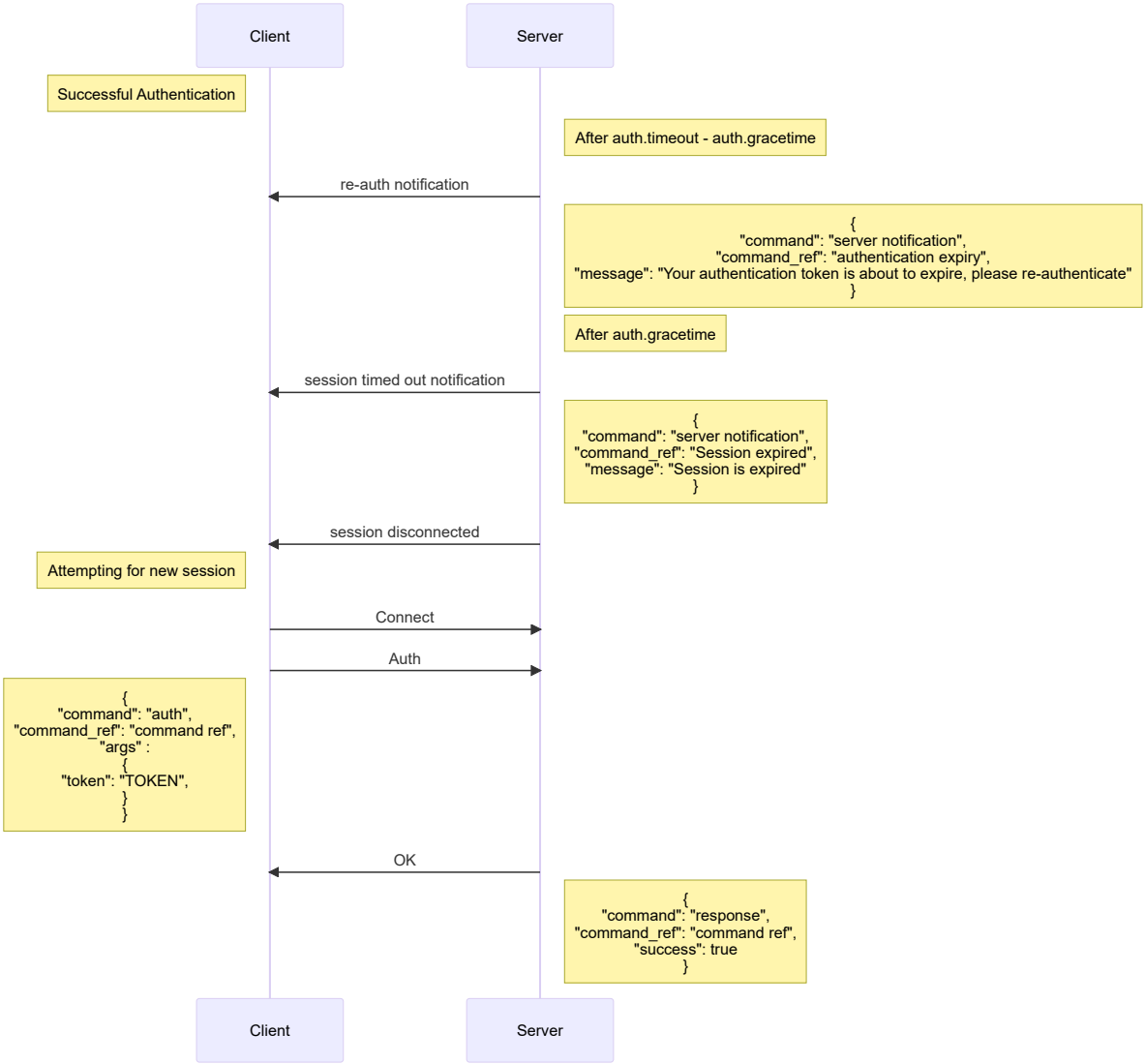
If client tries to get re-authenticated before receiving server re-authentication notification with same user token, server will sends "User already authenticated" error message to client.




## Case 4 : Client fails to re-auth itself during Gracetime

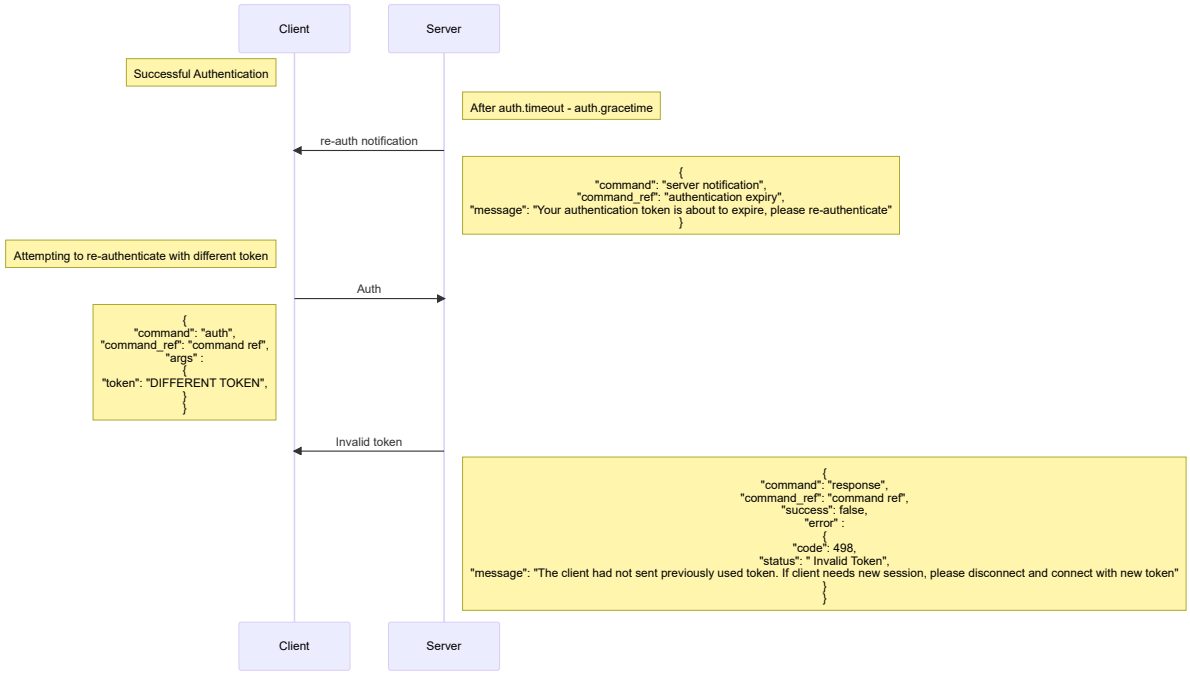
application.global.wba.auth.timeout = 60
application.global.wba.auth.gracetime = 30
Grace Time(60-30) = 30

If client did not re-authenticate within grace period (30 seconds) after receiving server re-authentication notification, server will send session timed out notification and client session will be disconnected. Client needs to connect to websocket again and start the new Authentication process.




## Case 5 : Client re-authenticates with different token

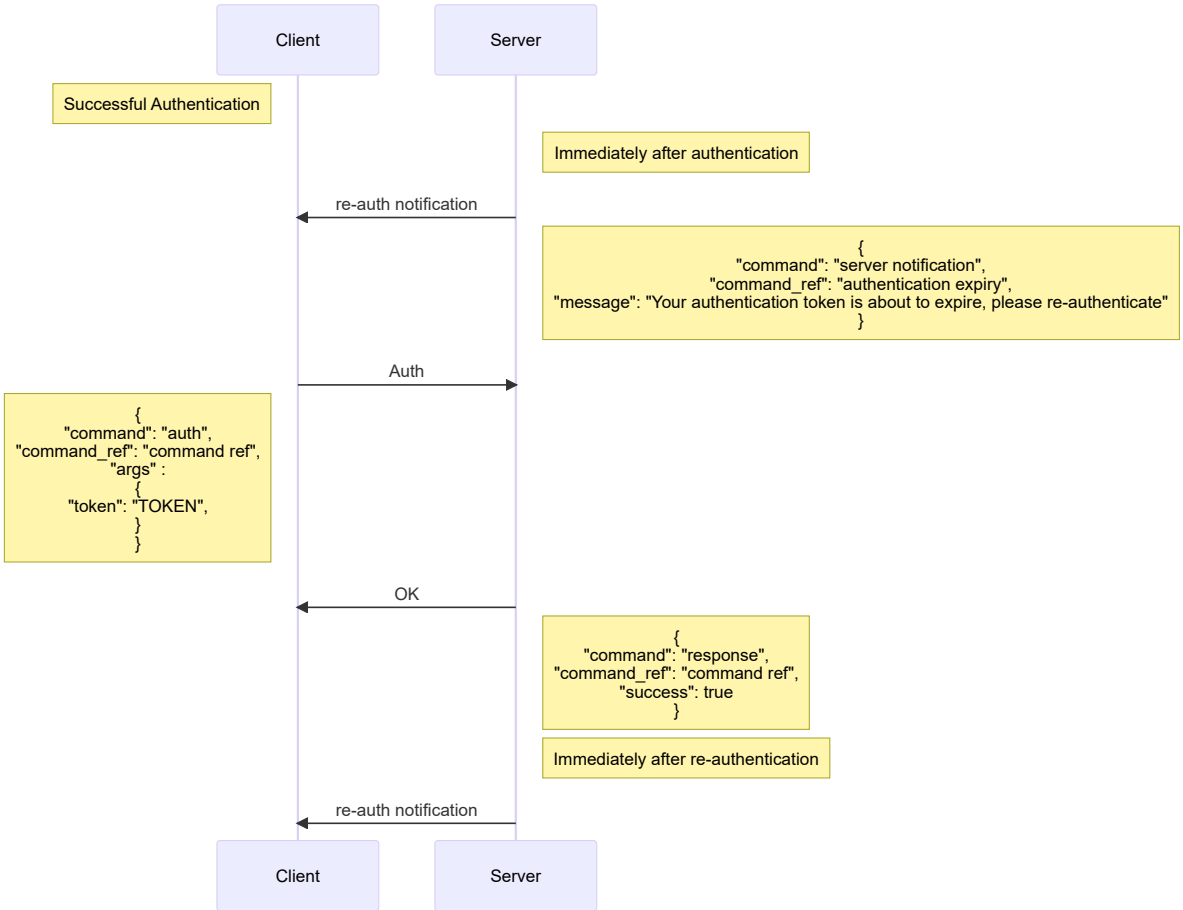
If client tries to re-authenticate within grace period (30 seconds) with a different token after receiving server re-authentication notification, server will sends invalid token message to client.




## Case 6 : Settings Auth Gracetime  is greater than  Auth Timeout

application.global.wba.auth.timeout = 20
application.global.wba.auth.gracetime = 30

This is not an expected configuration. However server will try to handle this. In this scenario, the server will send re-auth notification immediately after the client has authenticated the session and waits for grace period (30 seconds) to complete, if client re-authenticates within grace period (30 seconds), server will send again re-auth notification immediately after re-authentication and the process repeats.



## Authentication Responses

Various responses from the server for authentication/re-authentication are shown as below,

- After the successful Authentication , at the start of the GraceTime [AUTH_TIMEOUT - AUTH_GRACE_TIME] , the client will receive a re-authentication notification from server as shown below,

```
{
   "command": "server notification",
   "command_ref": "authentication expiry",
   "message": "Your authentication token is about to expire, please re-
authenticate"
}
```

- If Client reauths to re-authentication notification within the grace time period, the client session will be refreshed,

```
{
   "command": "response",
   "command_ref": "command reference",
   "success": true,
}
```

- If client reauthenitcates with different token than the one used for initial authentication, server will respond with the following error

```
{
   "command": "response",
   "command_ref": " command reference ",
   "success": false,
   "error":
      {
      "code": 498,
      "status": " Invalid Token ",
      "message": " The client had not sent previously used token. If client needs
new session, please disconnect and connect with new token "
      }
}
```

- If Client fails to reauthenticate within Gracetime, the client session will be disconnected and all the subscriptions are cleared and server will send a notification as shown below

```
{
  "command": "server notification",
   "command_ref": "Session expired",
   "message": "Session is expired"
}
```

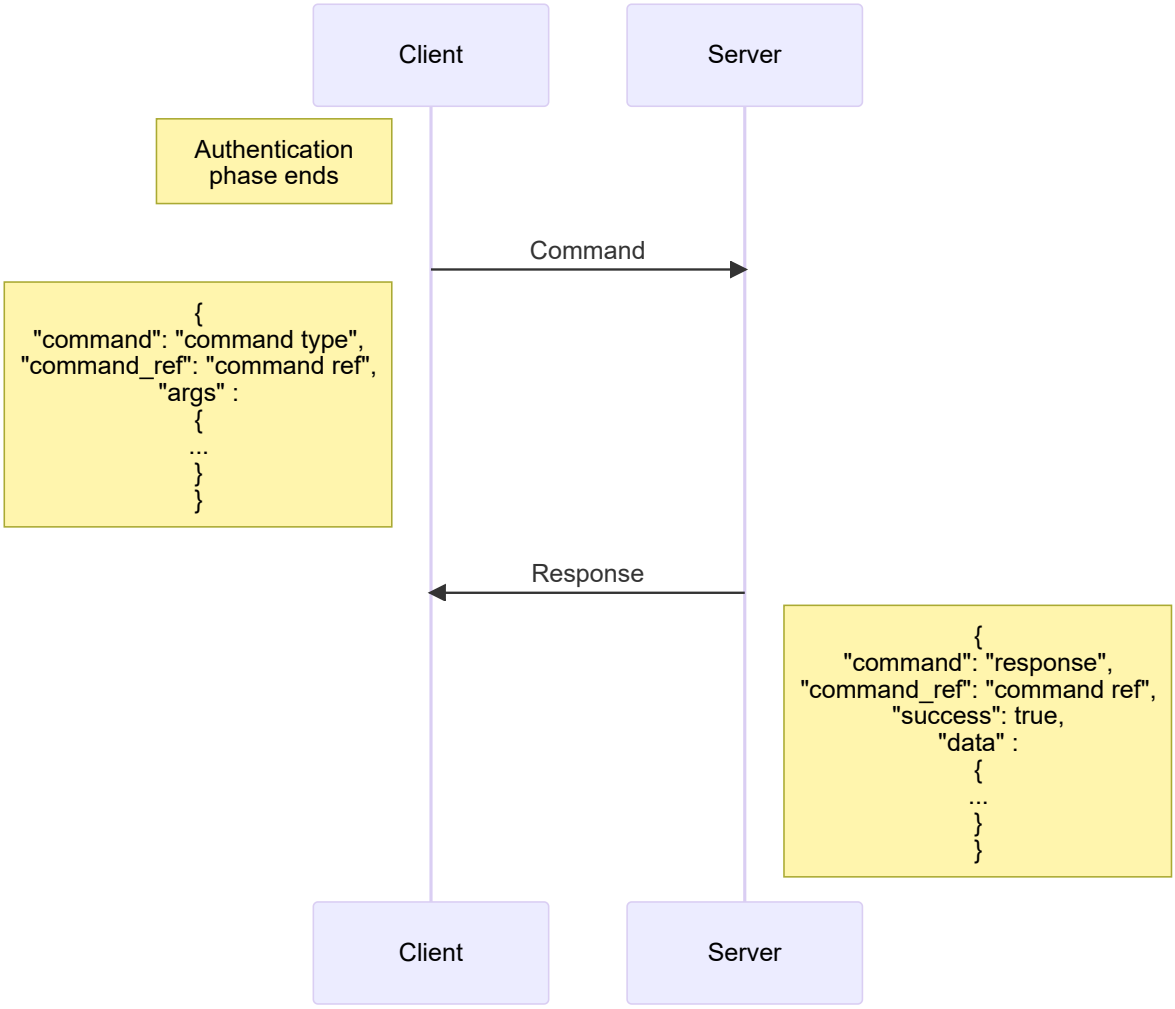
## Errors

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Missing mandatory parameter: token |
| 500 | Server Error | Internal Server Error | Data provisioning still in progress - Please try again later |
| 500 | Server Error | Internal Server Error | User already authenticated |
| 500 | Server Error | Internal Server Error | Another user is already connected with this token |
| 404 | Not Found | Resource Not Found | No user found for this token |

# Commands

The TradeSense WebSocket API commands allows to:

- fetch data from the Assure database
- call a service (e.g. CTD)
- subscribe to notifications




After successfully completing the authentication phase the client is allowed to send commands to the Server:

```
{
   "command":"command type",
   "command_ref":"command reference",
   "args":{}
}
```

Mandatory:

- `command` : type of the command;
- `command_ref` : the reference of the command is a string value generated by the client-side. This reference is used to identify the response linked to client's request;

Optional:

- `args` : command arguments.

## Fetch data

Available commands:

- `get_zones`
- `get_turrets`
- `get_users`
- `get_events`
- `get_calls`
- `get_version`
- `get_tpos`
- `get_lines`
- `get_health`
- `get_health_api_report`

### Get Zones

Retrieves data for all the Zones monitored by Assure.

```
{
    "command":"get_zones",
    "command_ref":"command reference",
    "args":{}
}
```

The response from the server will contain a list of Zone objects.

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{
      "zones":[
         {
            "id":"003O90001",
            "name":"Zone Assure (emtpy)",
            "locationCountryCode":null,
            "locationRegion":null,
            "locationCity":null,
            "callPrefix":null
         },
         {
            "id":"003NV0001",
            "name":"Assure (Verba)",
            "locationCountryCode":"BE",
            "locationRegion":"Province de Liège",
            "locationCity":"Huy",
            "callPrefix":"04"
         }
      ]
```

```
    }
}
```

**Optional arguments**

```
{
    "command":"get_zones",
    "command_ref":"command reference",
    "args":
    {
        "search": "value",
        "operator": "contains",
        "limit": response limit
    }
}
```

- `search` : search (string)
- `operator` : specify search operator (string)
- `limit` : limit the number of records returned (integer)

If `operator` filter is used, then `search` filter must be used. By using other than the above filters, server throws error message.

Available operators are `is_exactly`, `contains`, `begins_with`. Default is `contains`. By using other than these operators, server throws error message.

The TradeSense WBA will return all the entries which satisfies the search arguments.

The scope of the search is limited on:

- Zone name

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter search is not a valid string |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter operator is not a valid string |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter limit is not a valid integer |

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | one or more invalid filter identified |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | operator filter must be used along with search filter |

## Get Turrets

Retrieves data for all the Turrets monitored by Assure.

```
{
   "command":"get_turrets",
   "command_ref":"command reference",
   "args":{}
}
```

The response from the server will contain a list of Turret objects.

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{
      "turrets":[
         {
            "name":"T4-LOADTEST1",
            "ipAddress": "10.221.13.2",
            "alive":true,
            "tssVersion":"default",
            "zone":"Assure (Verba)",
            "username": "612341089",
            "firmwareVersion":"R9.3_5.50278",
            "recordingServer": {
               "vendor": "VERBA",
               "ipAddressPrimary": "10.221.7.147",
               "ipAddressSecondary": null
            },
            "firmwareTargetVersion":null,
            "model":"1015"
         },
         {
            "name":"T4-LOADTEST2",
            "ipAddress": "10.221.13.12",
            "alive":true,
            "tssVersion":"default",
            "zone":"Assure (Verba)",
```

```
            "username": null,
            "firmwareVersion":"R9.3_5.50278",
            "recordingServer": null,
            "firmwareTargetVersion":null,
            "model":"1015"
        }
    ]
  }
}
```

> The property "firmwareTargetVersion" is the target version for upgrade. This property will be set only pending upgrade process.

**Optional arguments**

```
{
    "command":"get_turrets",
    "command_ref":"command reference",
    "args":
    {
        "search": "value",
        "operator": "begins_with",
        "limit": response limit
    }
}
```

- `search` : search (string)
- `operator` : specify search operator (string)
- `limit` : limit the number of records returned (integer)

If `operator` filter is used, then `search` filter must be used. By using other than the above filters, server throws error message.

Available operators are `is_exactly`, `contains`, `begins_with`. Default is `contains`. Default is `contains`. By using Other than these operators, server throws error message.

The TradeSense WBA will return all the entries which satisfies the search arguments.

The scope of the search is limited on:

- Turret name

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter search is not a valid string |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter operator is not a valid string |

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter limit is not a valid integer |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | one or more invalid filter identified |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | operator filter must be used along with search filter |

## Get Users

Retrieves data for all the Users linked to the Zones managed by Assure.

```
{
   "command":"get_users",
   "command_ref":"command reference",
   "args":{}
}
```

The response from the server will contain a list of Users objects with their lines.

The Place info null is expected and in such cases we need to check the Source(TSS) information whether the configuration is proper or not. If the line configured to the user has a place associated.

Get Users return a large number of results as a list, WBA will page the result set in batch of certain size (example: 100, 200).

Note: Batch can be configured. Please refer Get Response batch size settings.

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{
      "current_batch": 1,
      "last_batch": 1,
      "users":[
         {
            "login":"612341089",
            "firstName":"IPTNetrix1",
            "lastName":"Turret1",
            "turret": "T4-LOADTEST1",
            "zone":"Assure",
```

```
            "status":"ACTIVE",
            "primaryLine":"+99999993201",
            "recordingServer": {
               "vendor": "VERBA",
               "ipAddressPrimary": "10.221.7.147",
               "ipAddressSecondary": null
            },
            "profile":{
               "lines":[
                  {
                     "userExtension": "+99999304300",
                     "userDisplayName": "PW 3043",
                     "tpoDnsName": "dnsName",
                     "subscriptionState": "SUBSCRIBED",
                     "defaultGlobalLine": false,
                     "voiceRecording": true,
                     "place":{
                         "name": "placeName",
                         "type": "RINGDOWN_DYNAMIC",
                         "state": "STARTED",
                         "cluster": "clusterName",
                         "groupId": 1234
                     },
                     "slots": [
                         {
                             "slot": "304301",
                             "label": "ARD 304301",
                             "type": "PERMANENT",
                             "target": "304301@Vega400G02",
                             "mrd": true,
                             "ard": "FIRST_USER",
                             "answer_only": null (or true/false)
                         },
                         {
                             "slot": "304302",
                             "label": "MRD 304302",
                             "type": "PERMANENT",
                             "target": "304302@Vega400G02",
                             "mrd": true,
                             "ard": "ALWAYS",
                             "answer_only": null (or true/false)
                         }
                     ],
                     "virtualSlotStart": null,
                     "virtualSlotEnd": null
                  },
               ]
            }
         }
      ]
   }
}
```

**Optional arguments**

```
{
    "command":"get_users",
    "command_ref":"command reference",
    "args":
    {
        "search": "value",
        "operator": "is_exactly",
        "limit": response limit,
        "get_lines_info": false/true
    }
}
```

- `search` : search (string)
- `operator` : specify search operator (string)
- `limit` : limit the number of records returned (integer)
- `get_lines_info` : limit profile lines information (boolean)

If `operator` filter is used, then `search` filter must be used. By using other than the above filters, server throws error message.

Available operators are `is_exactly`, `contains`, `begins_with`. Default is `contains`. Default is `contains`. By using Other than these operators, server throws error message.

If the client sets `get_lines_info` to false, then the user information will be fetched without profile lines information.

If the client sets `get_lines_info` to true, then the user information will be fetched along with profile lines information.

By default the `get_lines_info` is set to true.

The TradeSense WBA will return all the entries which satisfies the search arguments.

The scope of the search is limited on:

- User login
- User firstname
- User lastname

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter search is not a valid string |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter operator is not a valid string |

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter limit is not a valid integer |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | one or more invalid filter identified |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | operator filter must be used along with search filter |

## Get Events

### Limitation

To prevent performance degradation over time, the application will only return the most recent records. The default value is set to 2 days and can be overridden in application settings.

- application.global.wba.db.fetch.max.interval

When no filters used like the below mentioned query client-side is allowed to retrieve the latest events. The default records limit value is set to 500 and can be overridden in application settings. The maximum default limit value can be set is 500

- application.global.wba.response.default.records.limit

Example: If we have 1000 entries of events data, When requested with the below query it would fetch the entries in the range of 501 to 1000.

```
{
   "command":"get_events",
   "command_ref":"command reference",
   "args":{
      "category":"event category"
   }
}
```

When filters are used like in the below mentioned query the client side is allowed to retrieve all events within the specified filter range from older to the most recent events. The maximum records limit value is set to 10,000.

- application.global.wba.response.max.records.limit

Note: If the filter range exceeds 10,000 the response data will return with a warning that maximum limit exceeded in the status field after retrieving 10,000 entries.

#### Using ID

The client-side are allowed to retrieve all the events by categories. Indicate the segment of events using `from_id` and `to_id` keys. If `to_id` is not specified then all the events from indicated id will be sent.

```
{
   "command":"get_events",
   "command_ref":"command reference",
   "args":{
      "category":"event category",
      "from_id":event_id,
      "to_id":event_id
   }
}
```

- `category` : events categories that the third party request
  Possible values :
  - `calls`
  - `presences`
- `from_id` : the id of the first element of the batch that the third party request
- `to_id` : the id of the last element of the batch that the third party request

#### Using datetime

The client-side are also allowed to retrieve events by datetime. Indicate the segment of calls using `start_time` and `end_time` keys.

```
{
   "command":"get_events",
   "command_ref":"command reference",
   "args":{
      "category":"event category",
      "start_time":"datetime",
      "end_time":"datetime"
   }
}
```

- `category` : events categories that the third party request
  Possible values :
  - `calls`
  - `presences`
- `start_time` : the datetime considered as start
- `end_time` : the datetime considered as end

**Response**

The response from the server will contain a list of Event objects.

Property "status" in data field is to know the status of the response from the server.

> The event with asked `from_id` will be considered as already received by the third party, and then will not be included in the returned batch. However the event with the `to_id` will be included in the batch.


```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{
      "current_batch":1,
      "last_batch":1,
      "events":[
          {
            "event_id": 35692,
            "turret": "DESKDEV-VMWS138",
            "zone": "test_zone",
            "login": "Hyma",
            "device": "HS1",
            "microphone_state": "ON",
            "local_extension": "778303",
            "ddi_slot_extension": "77833901",
            "remote_extension": "11903",
            "call_direction": "OUTGOING",
            "call_ref": "2147484647@77830302",
            "state": "Ringing",
            "sip_call_id": "484441a9-63aab247-1f0d27-87b18418",
            "local_sip_handle": "5d6c92c0-5e376249-277c31-ab709d0",
            "created_at": "2022-10-27T09:01:01Z",
            "conference_call_ids": []
          },
          {
            "event_id": 35694,
            "turret": "DESKDEV-VMWS138",
            "zone": "test_zone",
            "login": "Hyma",
            "device": "HS1",
            "microphone_state": "ON",
            "local_extension": "778303",
            "ddi_slot_extension": "77833901",
            "remote_extension": "11903",
            "call_direction": "OUTGOING",
            "call_ref": "2147484647@77830302",
            "state": "Connected",
            "sip_call_id": "484441a9-63aab247-1f0d27-87b18418",
            "local_sip_handle": "5d6c92c0-5e376249-277c31-ab709d0",
            "created_at": "2022-10-27T09:01:04Z",
            "conference_call_ids": []
          },
          {
            "event_id": 35695,
```

```
            "turret": "DESKDEV-VMWS138",
            "zone": "test_zone",
            "login": "Hyma",
            "device": "HS1",
            "microphone_state": "ON",
            "local_extension": "778303",
            "ddi_slot_extension": "77833901",
            "remote_extension": "11901",
            "call_direction": "OUTGOING",
            "call_ref": "2147484646@77830301",
            "state": "Merged",
            "sip_call_id": "184a8142-670a646c-1f0d1c-8354a218",
            "local_sip_handle": "1d445c85-3f0d29f3-277c22-ab304d0",
            "created_at": "2022-10-27T09:01:07Z",
            "conference_call_ids": [
              {
                "local_call_id": "2147484646@77830301"
              },
              {
                "local_call_id": "2147484647@77830302"
              }
            ]
          },
          {
            "event_id": 35696,
            "turret": "DESKDEV-VMWS138",
            "zone": "test_zone",
            "login": "Hyma",
            "device": null,
            "microphone_state": null,
            "local_extension": "778303",
            "ddi_slot_extension": "77833901",
            "remote_extension": "11903",
            "call_direction": "OUTGOING",
            "call_ref": "2147484647@77830302",
            "state": "Disconnected",
            "sip_call_id": "484441a9-63aab247-1f0d27-87b18418",
            "local_sip_handle": "5d6c92c0-5e376249-277c31-ab709d0",
            "created_at": "2022-10-27T09:01:07Z",
            "conference_call_ids": []
          }
      ],
       "status": "ok"
   }
}
```


Note: Batch size can be configured. Please refer Get Response batch size settings.

The current and the last batch number will be referred in the data object.


```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{
```

```
    "current_batch":10,
    "last_batch":10,
    "events":[
        {
          "event_id": 35692,
          "turret": "DESKDEV-VMWS138",
          "zone": "test_zone",
          "login": "Hyma",
          "device": "HS1",
          "microphone_state": "ON",
          "local_extension": "778303",
          "ddi_slot_extension": "77833901",
          "remote_extension": "11903",
          "call_direction": "OUTGOING",
          "call_ref": "2147484647@77830302",
          "state": "Ringing",
          "sip_call_id": "484441a9-63aab247-1f0d27-87b18418",
          "local_sip_handle": "5d6c92c0-5e376249-277c31-ab709d0",
          "created_at": "2022-10-27T09:01:01Z",
          "conference_call_ids": []
        },
        {
          "event_id": 35694,
          "turret": "DESKDEV-VMWS138",
          "zone": "test_zone",
          "login": "Hyma",
          "device": "HS1",
          "microphone_state": "ON",
          "local_extension": "778303",
          "ddi_slot_extension": "77833901",
          "remote_extension": "11903",
          "call_direction": "OUTGOING",
          "call_ref": "2147484647@77830302",
          "state": "Connected",
          "sip_call_id": "484441a9-63aab247-1f0d27-87b18418",
          "local_sip_handle": "5d6c92c0-5e376249-277c31-ab709d0",
          "created_at": "2022-10-27T09:01:04Z",
          "conference_call_ids": []
        },
        {
          "event_id": 35695,
          "turret": "DESKDEV-VMWS138",
          "zone": "test_zone",
          "login": "Hyma",
          "device": "HS1",
          "microphone_state": "ON",
          "local_extension": "778303",
          "ddi_slot_extension": "77833901",
          "remote_extension": "11901",
          "call_direction": "OUTGOING",
          "call_ref": "2147484646@77830301",
          "state": "Merged",
          "sip_call_id": "184a8142-670a646c-1f0d1c-8354a218",
          "local_sip_handle": "1d445c85-3f0d29f3-277c22-ab304d0",
          "created_at": "2022-10-27T09:01:07Z",
          "conference_call_ids": [
```

```
            {
              "local_call_id": "2147484646@77830301"
            },
            {
              "local_call_id": "2147484647@77830302"
            }
          ]
        },
        {
          "event_id": 35696,
          "turret": "DESKDEV-VMWS138",
          "zone": "test_zone",
          "login": "Hyma",
          "device": null,
          "microphone_state": null,
          "local_extension": "778303",
          "ddi_slot_extension": "77833901",
          "remote_extension": "11903",
          "call_direction": "OUTGOING",
          "call_ref": "2147484647@77830302",
          "state": "Disconnected",
          "sip_call_id": "484441a9-63aab247-1f0d27-87b18418",
          "local_sip_handle": "5d6c92c0-5e376249-277c31-ab709d0",
          "created_at": "2022-10-27T09:01:07Z",
          "conference_call_ids": []
        },
       "..."
    ],
     "status": "Maximum limit exceeded"
  }
}
```


Note: Events will always be in ascending order with respect to event id.

Note: For Call States information, Please refer Call States in Get Calls

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Missing mandatory parameter: category |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter category is not valid |

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter from_id is not a valid integer |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter to_id is not a valid integer |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter start_time is not a valid ISO8601 date |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter end_time is not a valid ISO8601 date |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Can't use to_id without a from_id parameter |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Can't use end_time without a start_time parameter |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Can't mix from_id/to_id with start_time/end_time |

## Get Calls

### Limitation

To prevent performance degradation over time, the application will only return the most recent records. The default value is set to 2 days and can be overrided in application settings.

- application.global.wba.db.fetch.max.interval

To improve performance while handling call processing events, a new setting is introduced. The default value is set to 500 which is configurable. This means a new cdr with same SIP call id would be created along with a linked_cdr_id after 500 operations on the call. The minimum value allowed is 100. If the value is set below minimum value, it will take default value.

- application.assure.cdr.max.allowed.operations = 500

When no filters used like the below mentioned query client-side is allowed to retrieve the latest calls. The default records limit value is set to 500 and can be overridden in application settings. The maximum default limit value can be set is 500

- application.global.wba.response.default.records.limit

Example: If we have 1000 entries of calls data, When requested with the below query it would fetch the entries in the range of 501 to 1000.

```
{
   "command":"get_calls",
   "command_ref":"command reference",
   "args":{}
}
```

When filters are used like in the below mentioned query the client side is allowed to retrieve all calls within the specified filter range from older to the most recent calls. The maximum records limit value is set to 10,000.

- application.global.wba.response.max.records.limit

Note: If the filter range exceeds 10,000 the response data will return with a warning that maximum limit exceeded in the status field after retrieving 10,000 entries.

### Using ID

The client-side are allowed to retrieve calls with corresponding events. Indicate the segment of calls using `from_id` and `to_id` keys. If `to_id` is not specified then all the calls from indicated id will be sent.

```
{
   "command":"get_calls",
   "command_ref":"command reference",
   "args":{
      "from_id":call id,
      "to_id":call id
   }
}
```

- `from_id` : the id of the first call of the batch that the third party request
- `to_id` : the id of the last call of the batch that the third party request
- `filter` : request filtering of returned calls depending on their state
  - `all` (default)
  - `in_progress`
  - `done`
  - `on_held`

#### Using datetime

The client-side are also allowed to retrieve calls by datetime. Indicate the segment of calls using `start_time` and `end_time` keys. Only calls **started** during the segment will be returned.

```
{
   "command":"get_calls",
   "command_ref":"command reference",
   "args":{
      "start_time":"datetime",
      "end_time":"datetime"
   }
}
```

- `start_time` : the datetime considered as start
- `end_time` : the datetime considered as end
- `filter` : request filtering of returned calls depending on their state
  - `all` (default)
  - `in_progress`
  - `done`
  - `on_held`

#### Using call_ref

The client-side are also allowed to retrieve a single call by its `call_ref`.

```
{
   "command":"get_calls",
   "command_ref":"command reference",
   "args":{
      "call_ref":"2147491577@601602"
   }
}
```

- `call_ref` : the requested call_ref

### Using filter

The client-side are also allowed to querry calls using filter.

```
{
   "command":"get_calls",
   "command_ref":"command reference",
   "args":{
        "filter":filter
   }
}
```

- `filter` : request filtering of returned calls depending on their state
  - `all`
  - `in_progress`
  - `done`
  - `on_held`

### Response

The response from the server will contain a list with the call object and its events.

Get Calls return a large number of results as a list, WBA will page the result set in batch of certain size (example: 100, 200).

Property "status" in data field is to know the status of the response from the server.

Note: Batch size can be configured. Please refer Get Response batch size settings

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{
      "current_batch":1,
      "last_batch":1,
      "calls":[
         {
            "call_id": 6908,
            "call_ref": "2147484646@77830301",
            "turret": "DESKDEV-VMWS138",
            "login": "Hyma",
            "local_extension": "778303",
            "remote_extension": "11901",
            "call_direction": "OUTGOING",
            "call_start_time": "2022-10-27T09:00:50Z",
            "call_end_time": "2022-10-27T09:01:10Z",
            "sip_call_id": "184a8142-670a646c-1f0d1c-8354a218",
            "local_sip_handle": "1d445c85-3f0d29f3-277c22-ab304d0",
            "ddi_slot_extension": "77833901",
            "linked_cdr_id": 234,
            "events": [
              {
                "event_id": 35688,
```

```
                "operation": "Ringing",
                "time": "2022-10-27T09:00:50Z",
                "device": "HS1",
                "conference_call_ids": [],
                "microphone_state": "ON"
              },
              {
                "event_id": 35690,
                "operation": "Connected",
                "time": "2022-10-27T09:00:53Z",
                "device": "HS1",
                "conference_call_ids": [],
                "microphone_state": "ON"
              },
              {
                "event_id": 35691,
                "operation": "Held_For_Transfer_Or_Conference",
                "time": "2022-10-27T09:00:56Z",
                "device": "HS1",
                "conference_call_ids": [],
                "microphone_state": "ON"
              },
              {
                "event_id": 35695,
                "operation": "Merged",
                "time": "2022-10-27T09:01:07Z",
                "device": "HS1",
                "conference_call_ids": [
                  {
                    "local_call_id": "2147484646@77830301"
                  },
                  {
                    "local_call_id": "2147484647@77830302"
                  }
                ],
                "microphone_state": "ON"
              },
              {
                "event_id": 35697,
                "operation": "Disconnected",
                "time": "2022-10-27T09:01:10Z",
                "device": null,
                "conference_call_ids": [],
                "microphone_state": null
              }
            ],
            "created_at": "2022-10-27T09:01:11Z",
            "tpo_dns_name": "office1.itsnet.bt.com"
          }
        ],
         "status": "ok"
    }
}
```

Note: With the specified filter range if number of data will be more than the limit set by the user then status will be maximum limit exceeded.

Note: calls will always be in ascending order with respect to call_id.

Possible values:

- `device` : turret audio device name where the call is actually hosted
  - `HS1`
  - `HS2`
  - `LD1.1`
  - `LD2.4`
- `microphone_state` : state of the microphone
  - `On`
  - `Off`

**Call States**

| Call State | Explanation |
|---|---|
| Ringing | An outbound or inbound call is Ringing |
| Connected | An outbound or inbound call is connected to a device (HS,LD,HF) |
| Disconnected | An outbound or inbound call is disconnected or terminated |
| Held | An outbound or inbound call is on Hold |
| Held_For_Transfer_Or_Conference | An outbound or inbound call is on Hold for Transfer Or Conference |
| Merged | An outbound or inbound call is in Merge Conference |
| IntercomDirectConnected | A Direct Intercom is connected with a remote party. No one is actively talking on it |
| IntercomDirectConnectedIn | A Direct Intercom is connected with a remote party. The remote party is actively speaking |
| IntercomDirectConnectedOut | A Direct Intercom is connected with a remote party. The turret user is actively speaking |
| IntercomDirectConnectedInOut | A Direct Intercom is connected with a remote party. The turret user and the remote party are both connected and actively speaking to each other |
| IntercomGroupConnected | A Group Intercom is connected |

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter from_id is not a valid integer |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter to_id is not a valid integer |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter start_time is not a valid ISO8601 date |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter end_time is not a valid ISO8601 date |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Filter parameter is not valid |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Can't use to_id without a from_id parameter |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Can't use end_time without a start_time parameter |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Can't mix from_id/to_id with start_time/end_time |

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Can't mix from_id/start_time with call_ref |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Can't mix filter with call_ref |

## Get Version

Retrieves versions for all the components of TradeSense.

```
{
   "command":"get_version",
   "command_ref":"command reference",
   "args":{}
}
```

### Response

The response from the server will contain the TradeSense components and their current versions.

```
{
   "command": "response",
   "command_ref": "command reference",
   "success": true,
   "data":{
      "BTPT-build": "10.0.0",
      "lizzie": "10.0.0",
      "skink": "10.0.0",
      "os-upgrade-level": "22",
      "assure": "10.0.0",
      "backendapi": "10.0.0",
      "gecko": "10.0.0",
      "haproxy": "10.0.0"
   }
}
```

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | one or more invalid filter identified |

## Get Tpos

Retrieves data for all the TPOs monitored by Tradesense Assure.

```
{
   "command":"get_tpos",
   "command_ref":"command reference",
   "args":{}
}
```

The response from the server will contain a list of TPO objects.

```
{
  "command": "response",
  "command_ref": "HFYqKybmndzf",
  "success": true,
  "data": {
   "tpos": [
      {
        "name": "Radio-TPO1",
        "alive": true,
        "tssVersion": "9.7.8.56509",
        "currentState": "ACTIVE",
        "ipAddress": "10.221.3.116",
        "tpoDnsName": "radio.tait.com",
        "recordingServerEnabled": false,
        "zone": "System",
        "clusterName": "OfficeCluster1",
        "recordingServer": {
          "vendor": "REDBOX",
          "ipAddressPrimary": "10.221.7.200",
          "ipAddressSecondary": null
        }
      }
    ]
  }
}
```

**Optional arguments**

```
{
    "command":"get_tpos",
    "command_ref":"command reference",
    "args":
    {
        "search": "value",
        "operator": "begins_with",
        "limit": response limit
    }
}
```

- `search` : search (string)
- `operator` : specify search operator (string)
- `limit` : limit the number of records returned (integer)

If `operator` filter is used, then `search` filter must be used. By using other than the above filters, server throws error message.

Available operators are `is_exactly`, `contains`, `begins_with`. Default is `contains`. By using Other than these operators, server throws error message.

The TradeSense WBA will return all the entries which satisfies the search arguments.

The scope of the search is limited on:

- TPO Name

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter search is not a valid string |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter operator is not a valid string |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter limit is not a valid integer |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | one or more invalid filter identified |

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | operator filter must be used along with search filter |

## Get Lines

Retrieves data for all the TPO Lines monitored by Tradesense Assure.

```
{
   "command":"get_lines",
   "command_ref":"command reference",
   "args":{}
}
```

The response from the server will contain a list of Line objects.


```
{
    "command": "response",
    "command_ref": "zCvW5a1soNrq",
    "success": true,
    "data": {
        "lines": [
            {
                "lineExtension": "778306",
                "lineName": "Kevin E 778306",
                "lineType": "DDI",
                "tpoDnsName": "office1.itsnet.bt.com",
                "voiceRecording": true,
                "place": {
                    "name": "PW 821161",
                    "type": "RINGDOWN_DYNAMIC",
                    "state": "STARTED",
                    "cluster": "OfficeCluster1",
                    "groupId": 1
                },
                "slots": [
                    {
                        "slot": "82116102",
                        "label": "GEN GEN Line 2",
                        "type": "PERMANENT",
                        "target": "82116102@Vega8211",
                        "mrd": true,
                        "ard": "FIRST_USER",
                        "answer_only": null
                    }
                ],
                "virtualSlotStart": "82116101",
                "virtualSlotEnd": "82116104"
            }
```

```
        ]
    }
}
```

**Optional arguments**

```
{
    "command":"get_lines",
    "command_ref":"command reference",
    "args":
    {
        "search": "value",
        "operator": "begins_with",
        "limit": response limit
    }
}
```

- `search` : search (string)
- `operator` : specify search operator (string)
- `limit` : limit the number of records returned (integer)

If `operator` filter is used, then `search` filter must be used. By using other than the above filters, server throws error message.

Available operators are `is_exactly`, `contains`, `begins_with`. Default is `contains`. By using Other than these operators, server throws error message.

The TradeSense WBA will return all the entries which satisfies the search arguments.

The scope of the search is limited on:

- Line Extension

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter search is not a valid string |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter operator is not a valid string |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter limit is not a valid integer |

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | one or more invalid filter identified |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | operator filter must be used along with search filter |

## Get Shared Profiles

Retrieves data of all Shared Profiles.

```
{
   "command":"get_shared_profiles",
   "command_ref":"command reference",
   "args":{}
}
```

The response from the server will contain a list of Shared Profile objects with their shared lines.

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{
      "sharedprofiles":[
         {
            "profile_id":"612341089",
            "name":"IPTNetrix1",
            "sharedlines": {
               "version": "000000001",
               "profile_id": "001JM0002",
               "profile_line_id": "102000096",
               "user_domain": "test",
               "user_extention":"20164",
               "user_display_name": "IPC SUN",
               "type": "DDI",
               "boardcasting_policy":"ALL_BUT_IN",
               "sip_contact":"000009",
               "sip_device_id":"000000210",
               "priority": "NONE",
               "tpo_dns_name_id":"000LZ0001",
               "subscription_state": "MONITORED",
               "default_DDI": "false",
               "default_global_line":"false",
               "queue_type":"NONE",
               "radio_call_event":"false",
               "voice_recording":"YES"
            }
         }
```

```
        ]
    }
}
```

**Optional arguments**

```
{
    "command":"get_shared_profiles",
    "command_ref":"command reference",
    "args":
    {
        "search": "value",
        "operator": "begins_with"
    }
}
```

- `search` : search (string)
- `operator` : specify search operator (string)

If `operator` filter is used, then `search` filter must be used. By using other than the above filters, server throws error message.

Available operators are `is_exactly`, `contains`, `begins_with`. Default is `contains`. By using Other than these operators, server throws error message.

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter search is not a valid string |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter operator is not a valid string |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter limit is not a valid integer |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | one or more invalid filter identified |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | operator filter must be used along with search filter |

## Get Health Api Report

Retrieves the health report of TradeSense.

```
{
   "command":"get_health_api_report",
   "command_ref":"command reference",
   "args":{}
}
```

### Response

The response from the server will contain the health report/information of Certificates, TSS, Zones, Clusters, TPOs, TPO Places, Turrets, Recording Servers, PBX Servers and Alerts in Tradesense .

Alerts data is optional, user should use below command to get Alerts data in get_health_api.

```
{
   "command":"get_health_api_report",
   "command_ref":"command reference",
   "args":{"include":"alerts"}
}
```

Note: Alerts data will be in batch size

```
{"command": "response",
  "command_ref": "command reference",
  "success": true,
  "data":{
    "reportName": "<Report Name>",
    "reportTime": "<Report Time ISO Format>",
    "tradeSenseNodeName": "<TradeSense Node Name>",
    "cluster_name": "<Cluster Name>",
    "platform": {
      "name": "<PLATFORM NAME>",
    "Certificates": {
          "CA":
            {
              "Issuer": "<Issuer Name>",
              "Issued On": "<Issued On Date Time(ISO Format)>",
              "Expiry Date": "<Expiry Date Time(ISO Format)>"
            },
            "Signed_Cert":
            {
              "Issuer": "<Issuer Name>",
              "Issued On": "<Issued On Date Time(ISO Format)>",
              "Expiry Date": "<Expiry Date Time(ISO Format)>"
            },
           "TSS_Cert":
            {
              "Issuer": "<Issuer Name>",
              "Issued On": "<Issued On Date Time(ISO Format)>",
```

```
            "Expiry Date": "<Expiry Date Time(ISO Format)>"
        }
    },
    "platformName": {
    "name": "<Platform Name>",
    "tss": {
        "primaryTSS": {
            "url": "<URL>",
            "updated": "<Last Updated Time(ISO Format)>",
            "memory": {
                "free": "<Free Memory Space in GB>",
                "used": "<Used Memory Space in GB>",
                "total": "<Total Memory Space in GB>"
            },
            "freeDiskSpace": "<Free Disk Space in GB>",
            "freeDBSpace": "<Free DB Space in GB>",
            "freeLogSpace": "<Free Log Space in GB>",
            "replicationState": "<Replication State>",
            "CACertExpiry": "<CA Certificate Expiry Time(ISO Format)>",
            "SignedCertExpiry": "<SIGNED Certificate Expiry Time(ISO
Format)>",
            "TSSCertExpiry": "<TSS Certificate Expiry Time(ISO Format)>"
        },
        "secondaryTSS": {
            "url": "<URL>",
            "updated": "<Last Updated Time (ISO Format)>",
            "memory": {
                "free": "<Free Memory Space in GB>",
                "used": "<Used Memory Space in GB>",
                "total": "<Total Memory Space in GB>"
            },
            "freeDiskSpace": "<Free Disk Space in GB>",
            "freeDBSpace": "<Free DB Space in GB>",
            "freeLogSpace": "<Free Log Space in GB>",
            "replicationState": "<Replication State>",
            "CACertExpiry": "<CA Certificate Expiry Time(ISO Format)>",
            "SignedCertExpiry": "<SIGNED Certificate Expiry Time(ISO
Format)>",
            "TSSCertExpiry": "<TSS Certificate Expiry Time(ISO Format)>"
        }
    },
    "zones": [
        {
            "name": "<Zone Name>",
            "tpoClusters": [
                {
                    "name": "<Cluster Name>",
                    "clusterRedunancy": "SessionPerisitancy/Failover",
                    "tpos": [
                        {
                            "name": "<TPO Name>",
                            "order": "<Order id>",
                            "groupId": "<Group id>",
                            "tpoRole": "Active/Passive",
                            "currentGroupId": "<Current Group id>",
                            "currentTpoRole": "Active/Passive",
```

```
                                "updated": "<Last Updated Time(ISO Format)>",
                                "statusOnTSS": "true/false",
                                "snmpConnection": "true/false",
                                "upTime": "String",
                                "memory": {
                                    "free": "<Free Memory Space in GB>",
                                    "used": "<Used Memory Space in GB>",
                                    "total": "<Total Memory Space in GB>"
                                },
                                "freeDiskSpace": "<Free Disk Space in GB>",
                                "heartbeatFailureCount": "<Count>",
                                "numberOfCalls": "<Number of Calls>",
                                "voiceRecording": {
                                    "voiceRecorder": "<Recorder Name>",
                                    "status": "up/down/disabled"
                                }
                            }
                        ],
                        "places": [
                            {
                                "placeName": "<Place Name>",
                                "placeStatus": "Started/Stopped/Error",
                                "errorReason": "null/valid reason"
                            }
                        ]
                    }
                ],
                "turrets": [
                    {
                        "name": "<Turret Name>",
                        "alive": "true/false",
                        "loggedOn": "true/false",
                        "recordingServerEnabled": "true/false"
                    }
                ]
            }
        ],
        "voiceRecorders": [
            {
                "name": "<Voice Recorder Name>",
                "primaryStatus": "Active/Error",
                "secondaryStatus": "Active/Error",
            }
        ],
        "pbxServers": [
            {
                "name": "<PBX Name>",
                "pbxNodeStatus": "Active/Error",
                "lastupdatetime": "<Last Updated Time (ISO Format)>"
            }
            ]
        },
    "tradesenseNodeHealth": {
    "timestamp": "<Timestamp>",
    "hostname": "<Tradesense Host Name>",
    "ipaddress": "<Tradesense Ip Address>",
```

```
      "cpuUsed": "<Percentage Of CPU Used>",
      "loadAverage": "<Percentage Of Average Load>",
      "memoryUsed": "<Percentage Of Memory Used>",
      "diskUsed": {
        "sda_sda1": "<Used Disk Percentage>",
        "sda_sda2": "<Used Disk Percentage>",
        "sdb": "<Used Disk Percentage>",
        "sdc": "<Used Disk Percentage>",
        "sdd": "<Used Disk Percentage>",
        "sde": "<Used Disk Percentage>"
      },
      "assure": {
        "mysql": {
          "memoryUsed": "<Percentage Of Memory Used>",
          "cpuUsed": "<Percentage Of CPU Used>"
        },
        "cpuUsed": "<Percentage Of CPU Used>",
        "servicesDetails": {
          "checks": [
            {
              "data": {
                "status": [
                  "<Running Status>",
                  "uptime <Up Time Value>"
                ]
              },
              "id": "calleventlistener",
              "status": "<Service State>"
            },
            {
              "data": {
                "status": [
                  "<Running Status>",
                  "uptime <Up Time Value>"
                ]
              },
              "id": "celery-alerting",
              "status": "<Service State>"
            },
            {
              "data": {
                "status": [
                  "<Running Status>",
                  "uptime <Up Time Value>"
                ]
              },
              "id": "celery-cdr_correlation",
              "status": "<Service State>"
            },
            {
              "data": {
                "status": [
                  "<Running Status>",
                  "uptime <Up Time Value>"
                ]
              },
```

```
            "id": "celery-cdr_process_call_events",
            "status": "<Service State>"
          },
          {
            "data": {
              "status": [
                "<Running Status>",
                "uptime <Up Time Value>"
              ]
            },
            "id": "celery-high-priority",
            "status": "<Service State>"
          },
          {
            "data": {
              "status": [
                "<Running Status>",
                "uptime <Up Time Value>"
              ]
            },
            "id": "celery-inventory",
            "status": "<Service State>"
          },
          {
            "data": {
              "status": [
                "<Running Status>",
                "uptime <Up Time Value>"
              ]
            },
            "id": "celery-inventory_turret_updates",
            "status": "<Service State>"
          },
          {
            "data": {
              "status": [
                "<Running Status>",
                "uptime <Up Time Value>"
              ]
            },
            "id": "celery-wtf",
            "status": "<Service State>"
          },
          {
            "data": {
              "status": [
                "<Running Status>",
                "uptime <Up Time Value>"
              ]
            },
            "id": "celerybeat",
            "status": "<Service State>"
          },
          {
            "data": {
              "status": [
```

```
            "<Running Status>",
            "uptime <Up Time Value>"
          ]
        },
        "id": "crond",
        "status": "<Service State>"
      },
      {
        "data": {
          "status": [
            "<Running Status>",
            "uptime <Up Time Value>"
          ]
        },
        "id": "httpd",
        "status": "<Service State>"
      },
      {
        "data": {
          "status": [
            "<Running Status>",
            "uptime <Up Time Value>"
          ]
        },
        "id": "linestateseventlistener",
        "status": "<Service State>"
      },
      {
        "data": {
          "status": [
            "<Running Status>",
            "uptime <Up Time Value>"
          ]
        },
        "id": "mysqld",
        "status": "<Service State>"
      },
      {
        "data": {
          "status": [
            "<Running Status>",
            "uptime <Up Time Value>"
          ]
        },
        "id": "redis",
        "status": "<Service State>"
      },
      {
        "data": {
          "status": [
            "<Running Status>",
            "uptime <Up Time Value>"
          ]
        },
        "id": "snmpeventlistener",
        "status": "<Service State>"
```

```
            },
            {
              "data": {
                "status": [
                  "<Running Status>",
                  "uptime <Up Time Value>"
                ]
              },
              "id": "snmptrapd",
              "status": "<Service State>"
            }
          ],
          "outcome": "<Service State>"
        },
        "memoryUsed": "<Percentage Of Memory Used>"
      },
      "gecko": {
        "cpuUsed": "<Percentage Of CPU Used>",
        "servicesDetails": {
          "checks": [
            {
              "id": "Provisioning Data",
              "status": "<Service State>",
              "data": {
                "status": [
                  "<Status>"
                ]
              }
            },
            {
              "id": "TLS - Gecko CA status",
              "status": "<Service State>",
              "data": {
                "status": [
                  "<CA-Certificate uploaded Status>",
                  "Issuer DN : <Issuer DN Details>",
                  "Validity : <Certificate Valid From & Certificate Valid Upto>",
                  "Subject DN : <Subject DN Details>",
                  "Signature algorithm : <Certificate Signature>",
                  "Serial number : <Certificate Serial Number>"
                ]
              }
            },
            {
              "id": "TLS - TSS secure connection",
              "status": "<Service State>",
              "data": {
                "status": [
                  "<TLS Handshake Status>",
                  "Issuer DN : <Issuer DN Details>",
                  "Validity : <TLS Validity>",
                  "Subject DN : <Subject DN Details>",
                  "Signature algorithm : <TLS Signature>",
                  "Serial number : <Serial Number>"
                ]
              }
```

```
        },
        {
          "id": "TSS",
          "status": "<Service State>",
          "data": {
            "status": [
              "<TSS Status>"
            ]
          }
        },
        {
          "id": "Inter-cluster Links",
          "status": "<Service State>",
          "data": {
            "status": []
          }
        },
        {
          "id": "Config",
          "status": "<Service State>",
          "data": {
            "status": [
              "version: <Version Number>",
              "node-name: <Node Name>",
              "cluster-name: <Cluster Name>",
              "TSS version: <TSS Version>",
              "TSS use SSL: True/False",
              "TSS port: <TSS Port>",
              "TSS primary host: <Primary TSS Url>"
            ]
          }
        },
        {
          "id": "Hazelcast Cluster",
          "status": "<Service State>",
          "data": {
            "status": [
              "Cluster is active",
              "Cluster time <Time>",
              "Cluster version <Version>",
              "Cluster members [10.0.1.34]",
              "Quorum-size: <Quorum Size>",
              "No quorum rule in force",
              "Enabled for container-deployment: true"
            ]
          }
        }
      ],
      "outcome": "<Service State>"
    },
    "memoryUsed": "<Percentage Of Memory Used>"
  },
  "landing": {
    "cpuUsed": "<Percentage Of CPU Used>",
    "memoryUsed": "<Percentage Of Memory Used>"
  },
```

```
      "bayawak": {
        "cpuUsed": "<Percentage Of CPU Used>",
        "memoryUsed": "<Percentage Of Memory Used>"
      },
      "grafana": {
        "cpuUsed": "<Percentage Of CPU Used>",
        "memoryUsed": "<Percentage Of Memory Used>"
      },
      "version_server": {
        "cpuUsed": "<Percentage Of CPU Used>",
        "memoryUsed": "<Percentage Of Memory Used>"
      },
      "haproxy": {
        "cpuUsed": "<Percentage Of CPU Used>",
        "memoryUsed": "<Percentage Of Memory Used>"
      },
      "backendapi": {
        "cpuUsed": "<Percentage Of CPU Used>",
        "memoryUsed": "<Percentage Of Memory Used>"
      }
    }
  }
}
    }
}
{
  "command": "response",
  "command_ref": "command reference",
  "success": true,
  "data": {
    "current_batch": "<CURRENT BATCH>",
    "last_batch": "<Last Batch>",
    "alerts": [
      {
        "alert_id": "<ALERT ID>",
        "alert_severity": "<ALERT SEVERITY>",
        "alert_alarm_name": "<ALARM NAME>",
        "alert_updated_time": "<UPDATED TIME>",
        "alert_message": "<MESSAGE>",
        "hostname": "<HOSTNAME>",
        "hostip": "<HOSTIP>"
      }
      ]
   }
}
```


**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Invalid Argument for Health Api Report |

# Call a service

Available services:

- `ctd` (Click To Dial)

## Click To Dial

Remotely performs call actions on a turret.

The current version supports the following actions:

- Place a new outbound call
- Accept an incoming call
- Hold an established call
- Resume a held call
- Terminate an opened call
- Dispatch an opened call
- Transfer an opened call
  - Attended mode
  - Unattended mode
- Make a conference call
- Remotely Logon/Logoff a user on turret

The server-side will respond to each command with an object `data` containing the `call_ref`. This response doesn't mean that the command are actually executed by the turret but that the command was successfully transmitted.

### Place a new outbound call

Initiating a new outbound call is requested with the following message:

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"place",
      "login":"user login",
      "remote_extension":"extension to call",
      "local_extension":"Line",
      "ddi_slot":"DDI slot extension",
      "device":"device"
   }
}
```

Mandatory:

- `action` : category must be `place`

- `login` : user login

Optional:

- `remote_extension` : remote target phone extension to contact
- `local_extension` : the extension of the line that will be used to host the call. If not set, then the turret will automatically select a Slot and/or line.
- `ddi_slot` : allows to set the DDI slot used to place the call, to join a DDI slot with parameter or with a call already running.
- `device` : The audio device that will be used to host the call: in the BTTrade turret audio device format (HS1,HS2, LD1.1, LD2.4, ...). If not specified, the turret uses the current selected or active handset.

The `remote_extension`, `local_extension` and `ddi_slot` are all optional but at least one parameter is necessary.

Permissible combinations:

- `remote_extension` alone
- `remote_extension` and `local_extension`
- `remote_extension` and `ddi_slot`
- `ddi_slot` alone

#### Limitation

Default settings will refresh data every couple of minute. In order to get real time data about line registration state, target user must be configured with :

> profile.setting.crm.linestate.enable = true

TPO place state is not updated in real time.

#### Local extension availability

Before placing the call, the application will check for local extension availability.
Possible errors are:

- No line found for local extension
- No line found for DDI slot
- No default or primary line available
- Local extension TPO place is not started.
- Local extension is not registered.
- Can't call remote_extension from the same extension

#### Outgoing call errors

An outgoing call can fail for practical reasons. In that case, an additional call event will be sent with the failure reason in the state field.

These additional call events are not saved in assure or WBA. They are only here to notify subscribers why the call cannot complete and to allow them to clear the call from the turret.

```
{
```

```
    "event_id": null,
    "turret": "TS-TURRET-01",
    "zone": "Belgium - Liege - FHO",
    "login": "fho3",
    "device": "HS1",
    "microphone_state": "OFF",
    "local_extension": "6202",
    "remote_extension": "6402",
    "call_direction": "OUTGOING",
    "call_ref": "2147483671@620202",
    "state": "Extension has no free slot available",
    "created_at": "2022-01-13T23:20:35Z"
}
```

Possible errors are:

- Extension is unreachable
- Extension has no free slot available

This is only available for turrets running firmware 9.5.12, 9.7.1 or later.

#### Response

The server will respond with an object `data` containing the `call_ref`. Subsequent actions or notifications will use that reference to identify the call. The server will also provide `local_extension` that will actually be used to host the call.

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{
      "action":"place",
      "login":"user login",
      "turret":"turret name",
      "call_ref":"call ref",
      "local_extension":"Line or DDI slot extension"
   }
}
```

### Accept an incoming call

Accepting an incoming call is requested with the following message:

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"accept",
      "login":"user login",
      "call_ref":"call ref",
      "device":"device"
   }
}
```

Mandatory:

- `action` : action must be `accept`
- `login` : user login
- `call_ref` : the reference of the call to accept

Optional:

- `device` : The audio device that will be used to host the call: in the BTTrade turret audio device format (HS1,HS2, LD1.1, LD2.4, ...). If not specified, the turret uses the current selected or active handset.

The user's line of the incoming call must be subscribed or monitored.

#### Hold an established call

Holding an established call is requested with the following message:

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"hold",
      "login":"user login",
      "call_ref":"call ref"
   }
}
```

Mandatory:

- `action` : action must be `hold`
- `login` : user login
- `call_ref` : the reference of the call to hold

#### Resume a held call

Resume a held call is requested with the following message:

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"resume",
      "login":"user login",
      "call_ref":"call ref"
   }
}
```

Mandatory:

- `action` : action must be `hold`
- `login` : user login
- `call_ref` : the reference of the call to hold

#### Terminate an opened call

Terminate an opened call is requested with the following message:

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"terminate",
      "login":"user login",
      "call_ref":"call ref"
   }
}
```

Mandatory:

- `action` : action must be `terminate`
- `login` : user login
- `call_ref` : the reference of the call to hold

#### Dispatch an opened call

Dispatching request will transfer an opened call to a specified target extension without follow-up. Requested with the following message:

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"dispatch",
      "login":"user login",
      "call_ref":"call ref",
      "remote_extension":"extension to call"
   }
}
```

Mandatory:

- `action` : action must be `dispatch`
- `login` : user login
- `call_ref` : the reference of the call to hold
- `remote_extension` : remote target phone extension to contact

### Transfer an opened call

With the Transfer feature, an ongoing call can be transferred to another party. This can be made in two different manner, `attended` or `unattended` mode.

#### Limitation

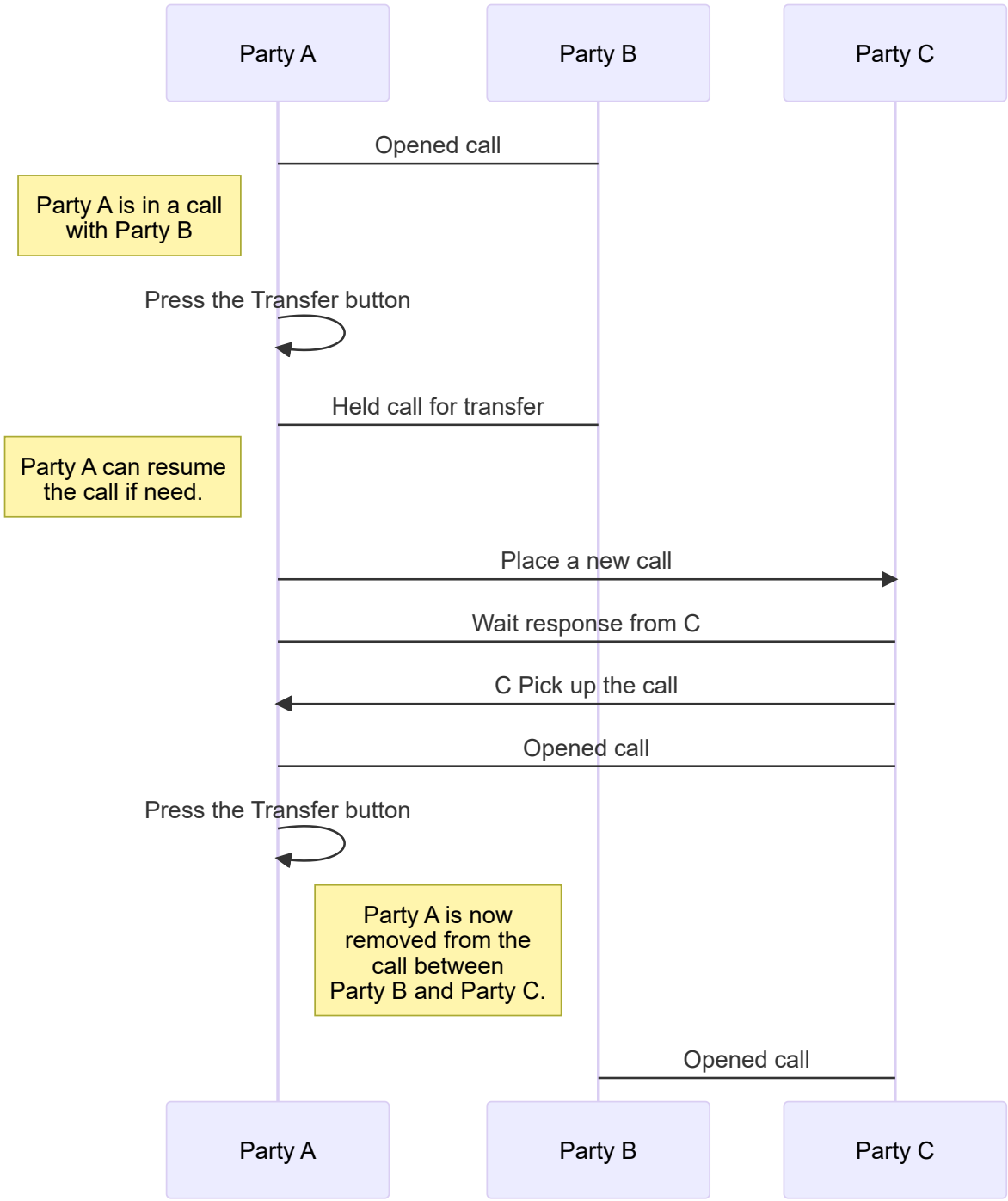
The transfer will only happen if:

- The two calls are both DDI calls or non DDI calls
- The two calls are established
- Unattended isn't supported

For transfer off DDI calls from a non-default DDI line, please be aware that the target turret must be configured with:

> profile.setting.ddi.advanced.handling.on.all = true

**Attended**




Let's say that Party A is in a call with Party B. To transfer the call to Party C, Party A will proceed as follow:

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"hold_transfer",
      "call_ref":"call ref"
   }
}
```

Established call with B must be held in special `hold_transfer` mode. Then establish a new call using action `new` and the same `local_extension` as the transferred call.

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"new",
      "login":"user login",
      "remote_extension":"extension to call",
      "local_extension":"Line or DDI slot extension"
   }
}
```

Mandatory:

- `action` : action must be `new` as you need to establish a new call before performing the transfer
- `login` : user login
- `remote_extension` : remote target phone extension to contact
- `local_extension` : the local extension must be the same as the one used for the transferred call

As for the place call command, local extension availability will be verified.

When Party C will pick-up the call. A notification will be sent to the client-side with the reference of the new call. To finish the transfer phase, send the action `perform_transfer` using both call ref.

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"perform_transfer",
      "login":"user login",
      "call_ref_source":"call ref",
      "call_ref_destination":"call ref"
   }
}
```

Mandatory:

- `action` : action must be `perform_transfer`
- `login` : user login
- `call_ref_source` :  the reference of the source call for transfer
- `call_ref_destination` : the reference of the target call

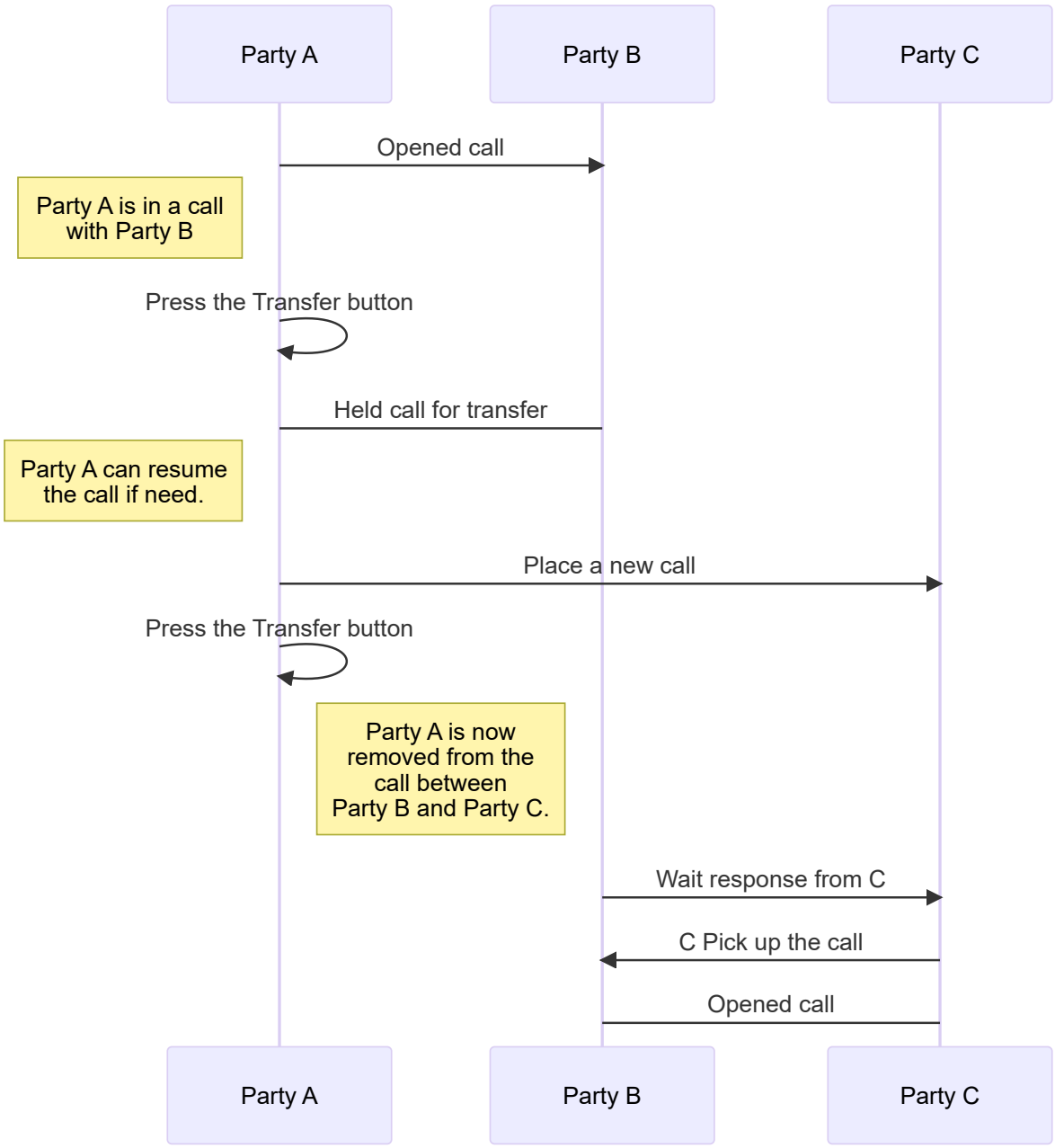
The source call for transfer is always the last call put on-hold with the `hold_transfer` mode. If this call is terminated or resumed, the transfer is not available anymore, even if there are other calls that have been previously put on-hold.

If the Party C do not pick up the call, Party A can resume the call and continue to talk to Party B.

Resume a held call is requested with the following message:

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"resume",
      "call_ref":"call ref"
   }
}
```

#### Unattended



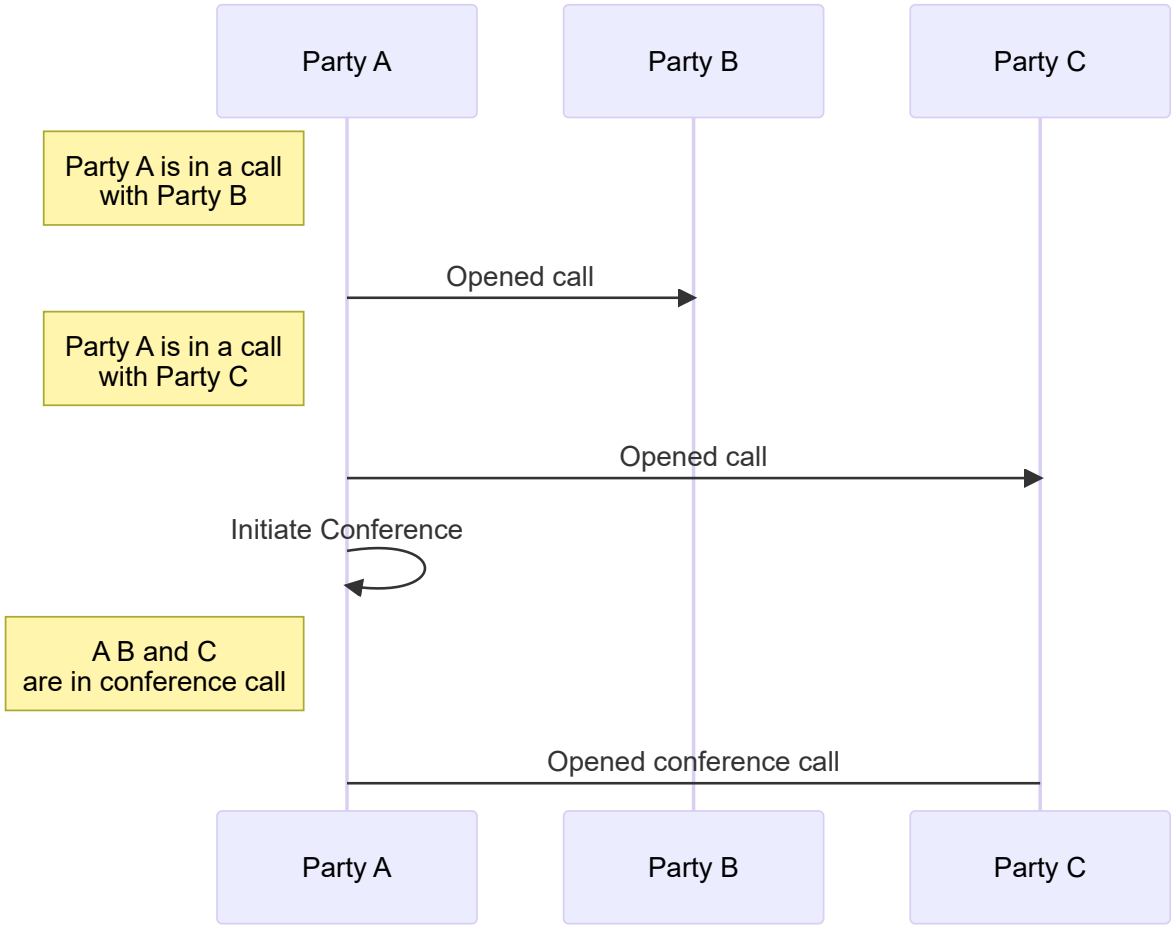

The only difference with the attended transfer is that Party A do not wait to establish the call and perform the transfer directly after placing the call to Party C. Thus the party A will be removed from the call and will not be able to resume it.

Currently This is not supported with the limitation on the turret end.

### Make a conference call

#### Limitation

> The conference will only happen if:
>
> - The two calls are DDI calls
> - The two calls are established
> - The two calls are both hosted on handset audio devices: ["HS1"; "HS2"]
> - The turret is configured to allow merging DDI calls (See profile.setting.ddi.conference.mode documentation)




Let's say A is in a call with B and with C, but B and C can't talk to each others. To start a conference, Party A should initiate a conference call.

Initiate a conference call is requested with the following message:

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"conference",
      "login":"user login",
      "call_ref_source":"call ref",
      "call_ref_destination":"call ref"
   }
}
```

Mandatory:

- `action` : action must be `conference`
- `login` : user login
- `call_ref_source` : the reference of the first call to put in the conference (here call between A and B)
- `call_ref_destination` : the reference of the second call to put in the conference (here call between A and C)

As for transfer, the conference must be performed on the same `local_extension`.
When the conference call will be established between all parties, only the `call_ref_source` will be kept. The `call_ref_destination` will be terminated. `call_ref_source` will then be able to host new participant.

### Remotely Logon/Logoff a user on turret

#### Limitation

To enable the Turret remote Logon/Logoff functionality:

Create a user with role as TSSAdmin in TSS ( console > Administrators )

Add below settings in TSS Productivity Tools cluster settings:

> application.global.wba.tss.api.user.login = TSSAdmin user name

> application.global.wba.tss.api.user.password = TSSAdmin user password

If the client sends Logon/Logoff command without adding above settings in TSS, then the server will send error response as shown below

```
{
  "command": "response",
  "command_ref": "command reference",
  "success": false,
  "error": {
    "code": 500,
    "status": "Server Error",
    "message": "Internal Server Error",
    "reason": "TSS authentication failed"
  }
}
```

#### Logon

To logon a user remotely, below request can be used

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"logon",
      "login":"user_name",
      "turret":"turret_name",
      "force":"True"
   }
}
```

Mandatory:

- `turret` : turret name (string)
- `action` : action must be `logon` (string)
- `login` : user name (string)
- `force` : (string) forcefully logon the user into turret, which takes either `True` or `False` (default - `False`)

The response from the server after successful logon is:

```
{
  "command": "response",
  "command_ref": "command reference",
  "success": true,
  "data": {
    "action": "logon",
    "turret": "turret_name",
    "login": "user_name",
    "force": "True"
  }
}
```

If the turret is already logged in with some other user and `force` is False in the command, then the response from the server is shown below

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"logon",
      "login":"user_name",
      "turret":"turret_name",
      "force":"False"
   }
}
```

```
{
  "command": "response",
  "command_ref": "command reference",
  "success": false,
  "error": {
    "code": 409,
    "status": "Conflict",
    "message": "User already logged on",
    "reason": "An user <user_name> is already logged on this turret"
  }
}
```

**Logoff**

To logoff a user, below request can be used

```
{
   "command":"service_ctd",
   "command_ref":"command reference",
   "args":{
      "action":"logoff",
      "logout":"user_name",
      "turret":"turret_name",
      "force": "False"
   }
}
```

Mandatory:

- `turret` : turret name (string)
- `action` : action must be `logoff` (string)

Optional:

- `logout`
- `force`

The response from the server after successful logoff is:

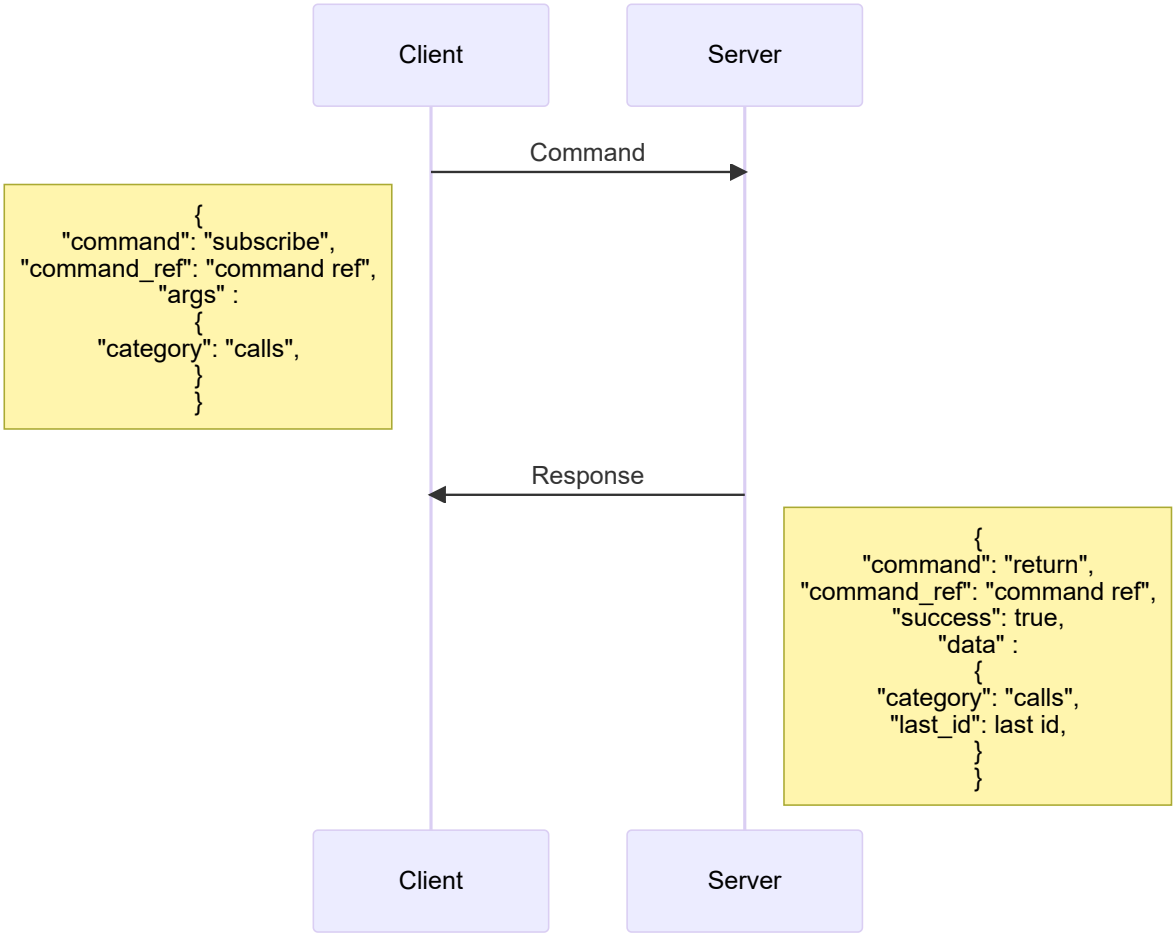
```
{
  "command": "response",
  "command_ref": "command reference",
  "success": true,
  "data": {
    "action": "logoff",
    "turret": "turret_name",
    "force": "False"
  }
}
```

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Command_ref already in use |
| 404 | Not Found | Resource Not Found | User not found |
| 500 | Server Error | Internal Server Error | User is not logged to any turret |
| 404 | Not Found | Resouce Not Found | No call found with this call_ref |
| 500 | Server Error | Internal Server Error | Unable to retrieve tradesense ip address |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | ClicktoDial: Can't use local_extension and ddi_slot at the same time |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | ClicktoDial: Can't use local_extension as only parameter |
| 404 | Not Found | Resource Not Found | No call found with this call_ref (source_call) |
| 404 | Not Found | Resource Not Found | No call found with this call_ref (destination_call) |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Missing mandatory parameter |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Parameter is not a valid integer |
| 404 | Not Found | Resource Not Found | No line found for local extension |

| Code | Status | Message | Reason |
|---|---|---|---|
| 404 | Not Found | Resource Not Found | No line found for DDI slot |
| 412 | Bad Request | Pre Condition Failed | No default or primary line available |
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Can't call remote_extension from the same extension |
| 500 | Server Error | Internal Server Error | Local extension TPO place is not started |
| 500 | Server Error | Internal Server Error | Local extension is not registered |
| 500 | Server Error | Internal Server Error | Unable to place a call |
| 500 | Server Error | Internal Server Error | Unable to terminate the call |
| 500 | Server Error | Internal Server Error | Unable to accept the call |
| 500 | Server Error | Internal Server Error | Unable to resume the call |
| 500 | Server Error | Internal Server Error | Unable to hold the call |
| 500 | Server Error | Internal Server Error | Unable to dispatch the call |
| 500 | Server Error | Internal Server Error | Unable to transfer the call |
| 500 | Server Error | Internal Server Error | Unable to merge the calls |
| 500 | Server Error | Internal Server Error | This user is already logged on this turret |
| 500 | Server Error | Internal Server Error | No user logged on this turret |
| 500 | Server Error | Internal Server Error | TSS authentication failed |
| 409 | Conflict | User already logged on | An user (user_name) is already logged on this turret |

## Subscribe to Notifications




The command `subscribe` will subscribe your client-side to the event bus. You can either listen to all events or to a specific event category. If you want to listen to multiple event categories, you will have to send multiple `subscribe` commands.

```
{
   "command":"subscribe",
   "command_ref":"command reference",
   "args":{
      "category":"events category"
   }
}
```

Mandatory:

- `category` : the category of events to subscribe on

The server will respond with a result message to indicate that the subscription is active.

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{
      "category":"calls",
      "last_id":last event id
   }
}
```

Client-side will receive the last event id available on server-side. If this id is different from the last received event, the client is able to request the server to re-transmit all the missed events (see Get Events).

Available notification categories:

- `calls` : receive all the notification for all alive calls and their state evolution
- `presences` : receive all the notification for presence state on turret.
- `alerts` : receive all the notification for Assure alerts.

### Errors

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request which includes requests with malformed JSON or invalid data values as well as requests in which the URL contains incorrect values | Missing mandatory parameter: category |
| 400 | Bad Request | The client has made a bad request which includes requests with malformed JSON or invalid data values as well as requests in which the URL contains incorrect values | Unknown parameter provided for: category |

## Unsubscribe to Notifications

The command `unsubscribe` will unsubscribe your client-side from the event notification. If you want to stop listening to multiple event categories, you will have to send multiple `unsubscribe` commands.

```
{
   "command":"unsubscribe",
   "command_ref":"command reference",
   "args":{
      "category":"events category"
   }
}
```

Mandatory:

- `category` : the category of events to unsubscribe on

The server will respond with a message to indicate the result of the `unsubscribe` command.

```
{
   "command":"response",
   "command_ref":"command reference",
   "success":true,
   "data":{
      "category":"calls",
      "message": "Client unsubscribed"
   }
}
```

Available notification categories:

- `calls` : stop receving calls notifications
- `presences` : stop receving presences notifications

# Notifications

In order to avoid an overflow of notifications, TradeSense API sends notifications grouped by sets. When a client-side is subscribed, server-side will send a message every second with all last notifications received.

## Calls notifications

TradeSense API allows third party to obtain contextual information to be eventually displayed when a call is either on the active handset, in ringing state or in held state. To be able to monitor all alive calls and their state evolution.

```
{
  "command": "notify",
  "data": {
    "category": "calls",
    "events": [
      {
        "event_id": 38572,
        "turret": "DESKDEV-VMWS138",
        "zone": "System",
        "login": "Hyma",
        "device": null,
        "microphone_state": null,
        "local_extension": "778303",
        "remote_extension": "11901",
        "call_direction": "OUTGOING",
        "call_ref": "2147484793@77830301",
        "state": "Disconnected",
        "sip_call_id": "4ecfbf2b-62bbe5d8-20a1d3-885f6518",
        "ddi_slot_extension": "77833901",
        "local_sip_handle": "40bd2caf-7449bf0c-2910d8-ab144d0",
        "created_at": "2022-10-28T13:47:43Z",
        "conference_call_ids": [],
        "reason": null,
        "reason_code": 0
      }
```

```
    ]
  }
}

{
  "command": "notify",
  "data": {
    "category": "calls",
    "events": [
      {
        "event_id": 38571,
        "turret": "DESKDEV-VMWS138",
        "zone": "System",
        "login": "Hyma",
        "device": "HS1",
        "microphone_state": "ON",
        "local_extension": "778303",
        "remote_extension": "11901",
        "call_direction": "OUTGOING",
        "call_ref": "2147484793@77830301",
        "state": "Connected",
        "sip_call_id": "4ecfbf2b-62bbe5d8-20a1d3-885f6518",
        "ddi_slot_extension": "77833901",
        "local_sip_handle": "40bd2caf-7449bf0c-2910d8-ab144d0",
        "created_at": "2022-10-28T13:47:41Z",
        "conference_call_ids": [],
        "reason": null,
        "reason_code": 0
      }
    ]
  }
}

{
  "command": "notify",
  "data": {
    "category": "calls",
    "events": [
      {
        "event_id": 14364,
        "turret": "DESKDEV-VMWS138",
        "zone": "Office",
        "login": "Hyma",
        "device": null,
        "microphone_state": null,
        "local_extension": "778303",
        "remote_extension": "11901",
        "call_direction": "OUTGOING",
        "call_ref": "2147483886@77830301",
        "state": "Disconnected",
        "sip_call_id": "4448ddf8-2e3e7921-1243fa-85a91f18-7303dd0a-13c4-759",
        "ddi_slot_extension": "77833901",
        "local_sip_handle": "5d3b6a60-34bbf169-1f5919-ac178d0-4600dd0a-13c4-759",
        "created_at": "2023-01-23T08:41:03Z",
        "conference_call_ids": [],
        "reason": "Session Expired",
```

```
        "reason_code": 440
      }
    ]
  }
}

{
  "command": "notify",
  "data": {
    "category": "calls",
    "events": [
      {
         "event_id": 123,
         "turret": "DESKDEV-VMWS138",
         "zone": "Office",
         "login": "Hyma",
         "device": "HS1",
         "microphone_state": "ON",
         "local_extension": "778303",
         "remote_extension": "1234",
         "call_direction": "UNKNOWN",
         "call_ref": "2147484471",
         "state": "ERROR",
         "sip_call_id": null,
         "ddi_slot_extension": "77833901",
         "local_sip_handle": "55276efe-3cf2e578-3460aa-c70bcd0-4600dd0a-13c4-
759",
         "created_at": "2023-02-08T07:28:50Z",
         "conference_call_ids": [],
         "reason": "is not a valid phone extension.",
         "reason_code": 1001
      }
    ]
  }
}

{
  "command": "notify",
  "data": {
    "events": [
      {
        "event_id": 254550,
        "turret": "vmazwebdev-33",
        "zone": "Office",
        "login": "Manasa",
        "device": "HFI1.1",
        "microphone_state": "ON",
        "local_extension": "778339",
        "remote_extension": "778304@10.221.23.31",
        "call_direction": "DIRECT-INTERCOM",
        "call_ref": "272",
        "state": "IntercomDirectConnectedOut",
        "sip_call_id": "426a8922-31372603-10e8-b3f64d0-1f17dd0a-13c4-759",
        "ddi_slot_extension": "77833901",
        "local_sip_handle": "426a8922-31372603-10e8-b3f64d0-1f17dd0a-13c4-759",
        "created_at": "2023-04-05T06:40:43Z",
```

```
        "conference_call_ids": [],
        "reason": null,
        "reason_code": 0
      }
    ],
    "category": "calls"
  }
}

{
  "command": "notify",
  "data": {
    "events": [
      {
        "event_id": 254531,
        "turret": "vmazwebdev-34",
        "zone": "Office",
        "login": "himavamsi",
        "device": "HFI1.1",
        "microphone_state": "ON",
        "local_extension": "778304",
        "remote_extension": "1886@office1.itsnet.bt.com",
        "call_direction": "GROUP-INTERCOM",
        "call_ref": "2147483745",
        "state": "INTERCOM-GROUP-ESTABLISHED",
        "sip_call_id": "22a842ef-1ac69115-100f-b2d58d0-1f17dd0a-13c4-759",
        "ddi_slot_extension": "77833901",
        "local_sip_handle": "22a842ef-1ac69115-100f-b2d58d0-1f17dd0a-13c4-759",
        "created_at": "2023-04-05T06:37:00Z",
        "conference_call_ids": [],
        "reason": null,
        "reason_code": 0
      }
    ],
    "category": "calls"
  }
}
```


All the notifications will contain following data:

- `event_id` : event id
- `turret` : turret name
- `zone` : turret's zone
- `login` : user login
- `device` : turret audio device name where the call is actually hosted
  - `HS1`
  - `HS2`
  - `LD1.1`
  - `LD2.4`
- `microphone_state` : state of the microphone

    - `ON`
    - `OFF`
    - null
- `local_extension` : local extension used for the call
- `remote_extension` : remote called extension
- `call_direction` : direction of the call
    - `OUTGOING`
    - `INCOMING`
    - `UNKNOWN`
    - `DIRECT-INTERCOM`
    - `GROUP-INTERCOM`
- `call_ref` : call reference
- `state` : the current state of the call
    - `Ringing`
    - `Connected`
    - `Held`
    - `Merged`
    - `Held Transfer`
    - `Held For Transfer Or Conference`
    - `Disconnected`
    - `Dispatched`
    - `Transferred`
    - `GroupIntercomConnected`
    - `IntercomDirectConnected`
    - `IntercomDirectConnectedIn`
    - `IntercomDirectConnectedOut`
    - `IntercomDirectConnectedInOut`
    - `Error`
- `created_at` : the exact time when the call switch to the indicated `state`
- `conference_call_ids` : local call ids that are in conference
- `reason` : failure reason for error events
- `reason_code` : reason codes for success and error events
    - `0` : success event
    - `1001` : is not a valid phone extension
    - `1002` : service unavailable
    - `1003` : phone extension can not be reached
    - `1004` : for unknown error events

- `440` : Session Expired

## Outgoing call scenario

If A want to establish a call with B. For an outgoing call TradeSense WBA will notify all the subscribers with following notifications:

> In this scenario we consider that there is 2 separate Zones monitored by 2 separate TradeSense WBA

- Third party on A side send a request to initiate a new call from A to B. Here we will use DDI slot id as local extension to specify the slot.

```
{
   "command":"service_ctd",
   "command_ref":"201",
   "args":{
      "action":"place",
      "login":"A",
      "remote_extension":"200",
      "local_extension":"10001"
   }
}
```

- A response will be sent back to notify if the request was successfully took into account

```
{
   "command":"response",
   "command_ref":"201",
   "success":true,
   "data":{
      "action":"place"
   }
}
```

- All the subscribers on B side events will be notified with an `incoming` call event. Multiple events messages will be sent if multiple users subscribe/monitor the line.

```
{
   "command":"notify",
   "data":{
      "category":"calls",
      "events":[
         {
            "event_id":1,
            "turret":"Turret_B",
            "zone":"Zone_Name",
            "login":"B",
            "device":"HS1",
            "microphone_state":"ON",
            "local_extension":"200",
            "remote_extension":"100",
            "call_direction":"INCOMING",
            "call_ref":"CALLREF01",
```

```
            "state":"Ringing",
            "created_at":"2020-05-07T18:13:03Z"
         }
      ]
   }
}
```

- Third party on B side send a request to accept the incoming call. Here we will use DDI slot id as local extension to specify the slot.

```
{
   "command":"service_ctd",
   "command_ref":"301",
   "args":{
      "action":"accept",
      "login":"B",
      "call_ref":"CALLREF01",
      "device":"HS1"
   }
}
```

- A response will be sent back to notify if the request was successfully took into account

```
{
   "command":"response",
   "command_ref":"301",
   "success":true,
   "data":{
      "action":"accept"
   }
}
```

- When B will pick up the call. All the subscribers on A and B side events will be notified with an `Connected` call event.

On side A:

```
{
   "command":"notify",
   "data":{
      "category":"calls",
      "events":[
         {
            "event_id":1,
            "turret":"Turret_A",
            "zone":"Zone_NameA",
            "login":"A",
            "device":"HS1",
            "microphone_state":"ON",
            "local_extension":"10001",
            "remote_extension":"200",
            "call_direction":"OUTGOING",
            "call_ref":"CALLREF01",
            "state":"Connected",
```

```
            "created_at":"2020-05-07T18:13:03Z"
         }
      ]
   }
}
```

On side B:

```
{
   "command":"notify",
   "data":{
      "category":"calls",
      "events":[
         {
            "event_id":1,
            "turret":"Turret_B",
            "zone":"Zone_NameB",
            "login":"B",
            "device":"HS1",
            "microphone_state":"ON",
            "local_extension":"20001",
            "remote_extension":"100",
            "call_direction":"INCOMING",
            "call_ref":"CALLREF01",
            "state":"Connected",
            "created_at":"2020-05-07T18:13:03Z"
         }
      ]
   }
}
```

- Third party on side A sends a request to terminate the call with B

```
{
   "command":"service_ctd",
   "command_ref":"202",
   "args":{
      "action":"terminate",
      "call_ref":"CALLREF01"
   }
}
```

- A response will be sent back to notify if the request was successfully took into account

```
{
   "command":"response",
   "command_ref":"202",
   "success":true,
   "data":{
      "action":"terminate"
   }
}
```

- As soon as the call is ended, all the subscribers on side A and B will be notified with an `Disconnected` call event

On side A:

```
{
   "command":"notify",
   "data":{
      "category":"calls",
      "events":[
         {
            "event_id":1,
            "turret":"Turret_A",
            "zone":"Zone_NameA",
            "login":"A",
            "device":"HS1",
            "microphone_state":"ON",
            "local_extension":"10001",
            "remote_extension":"200",
            "call_direction":"OUTGOING",
            "call_ref":"CALLREF01",
            "state":"Disconnected",
            "created_at":"2020-05-07T18:13:03Z"
         }
      ]
   }
}
```

On side B:

```
{
   "command":"notify",
   "data":{
      "category":"calls",
      "events":[
         {
            "event_id":1,
            "turret":"Turret_B",
            "zone":"Zone_NameB",
            "login":"B",
            "device":"HS1",
            "microphone_state":"ON",
            "local_extension":"20001",
            "remote_extension":"100",
            "call_direction":"INCOMING",
            "call_ref":"CALLREF01",
            "state":"Disconnected",
            "created_at":"2020-05-07T18:13:03Z"
         },
         "..."
      ]
   }
}
```

Note: For Call States information, Please refer Call States in Get Calls

### Error handling

If 401 Unauthorized error response is received, then client needs to re-authenticate and subscribe to notifications.

## Presence notifications

TradeSense API will notify all the subscribers with a presence message that the user has just performed a login or logout.

```
{
   "command":"notify",
   "data":{
      "category":"presences",
      "events":[
         {
            "event_id":event id,
            "turret":"turret name",
            "login":"user login",
            "zone":"turret's zone",
            "state":"presence state",
            "recordingServer": {
               "vendor": "vendor name",
               "ipAddressPrimary": "recording server primary ip address",
               "ipAddressSecondary": "recording server primary ip address"
            },
            "created_at":"datetime"
         },
         {
            "event_id":event id,
            "turret":"turret name",
            "login":"user login",
            "zone":"turret's zone",
            "state":"presence state",
            "recordingServer": null,
            "created_at":"datetime"
         }
      ]
   }
}
```

All the notifications will contain following data:

- `event_id` : event id
- `turret` : turret name
- `login` : user login
- `state` : the current state of the presence
- `created_at` : the exact time when the presence switched to the indicated `state`

### Presence state values

TradeSense API will notify all the subscribers with information about users presence on turrets.

The `state` value can be:

- `login` : user is logged-in
- `logout` : user is logged-out

## Alerts notifications

To enable Alerts notifications on WBA, the following setting should be added in the TSS Productivity Tools cluster settings:

> application.assure.alerts.enable.wba.notification = true

TradeSense API will notify all the subscribers with Assure alerts notifications as shown below

```
{
  "command": "notify",
  "data": {
    "category": "alerts",
    "events": [
      {
        "alert_id": "Alert Id",
        "alert_severity": "Alert Severity",
        "alert_alarm_name": "Alert Alarm Name",
        "alert_updated_time": "Alert Updated Time",
        "alert_message": "Alert Message",
        "hostname": "Hostname",
        "hostip": "Host IP Address"
      }
    ]
  }
}
```

All the alert notifications will contain following data:

- `alert_id` : alert id
- `alert_severity` : alert severity
- `alert_alarm_name` : alert alarm name
- `alert_updated_time` : alert updated time
- `alert_message` : alert message
- `hostname` : hostname
- `hostip` : host ip address

# Settings

It is possible to change default values by overriding the setting in the productivity tools clusters TSS page.

## Ping settings

> The server needs to be restarted for the changes to take effect

To keep the connection alive, a ping mechanism is enabled by default server-side.
A ping frame will be sent every N seconds.

**Enable the ping mechanism**

| Setting key | Default value |
|---|---|
| application.global.wba.ping.enabled | True |

**Configure the ping interval**

| Setting key | Default value |
|---|---|
| application.global.wba.ping.interval | 5 (in seconds) |

## Notifications settings

In order to avoid an overflow of notifications, WBA sends notifications grouped by batches. When a client-side is subscribed, server-side will send a message every N milliseconds with all last notifications received.

**Configure the alerts notification on WBA**

| Setting key | Default value |
|---|---|
| application.assure.alerts.enable.wba.notification | false |

## Get Response batch size settings

> The server needs to be restarted for the changes to take effect

In case of Get_Calls and Get_Events, the APIs might return a large data set depending on the request from client. TradeSense WBA will batch the data set and send the response to client request in batches. The size of this batch can be configured at TSS. The default value of the batch is 100.

In case of Get_Users, the APIs might return a large data set depending on the request from client. TradeSense WBA will batch the data set and send the response to client request in batches. The size of this batch can be configured at TSS. The default value of the batch is 50.

If the batch size is configured less than the default value, the batch size will be set to default. There is no upper limit on the batch size. Client needs to make an informed decision on increasing the batch value, as it might have performance issues.

**Response batch settings**

| Setting key | Default value |
|---|---|
| application.global.wba.response.batch.size | 100 |

| Setting key | Default value |
| --- | --- |
| application.global.wba.users.batch.size | 50 |

## Get Response limit settings

> The server needs to be restarted for the changes to take effect

We have limited the response data for both get_calls and get_events. By default the data will display the latest 500 calls/events when no filters are used and maximum 10,000 when filters are used.

**Response limit settings**

| Setting key | Default value |
| --- | --- |
| application.global.wba.response.default.records.limit | 500(Max value to be set) |

| Setting key | Default value |
| --- | --- |
| application.global.wba.response.max.records.limit | 10,000(Max value to be set) |

## Time out settings

As we don't have feedback from the turrets, when using the click to dial service, we need to monitor incoming events in order to be able to check if the request was executed.

**Configure the time out interval for click to dial requests**

This setting configure the time interval to wait for the event before sending an error message.

| Setting key | Default value |
| --- | --- |
| application.global.wba.ctd.monitoring.timeout | 10 (in seconds) |

**Configure the polling rate interval for click to dial monitoring**

This setting configure how often we check for timed out CTD requests.

| Setting key | Default value |
| --- | --- |
| application.global.wba.ctd.monitoring.poll.rate | 1 (in seconds) |

**Configure the time out interval for requests**

To avoid being overloaded by requests, incoming messages are only processed if not already timed out.
This setting configure the time interval to wait for a regular request to be timed out.

| Setting key | Default value |
| --- | --- |
| application.global.wba.request.timeout | 15 (in seconds) |

### Outdated data settings

#### Configure the fetch interval for WBA requests

To avoid too many records being returned, we set a maximum fetch interval in days. Calls/events older than N days won't be fetched from database.

| Setting key | |
|---|---|
| application.global.wba.db.fetch.max.interval | 2 (in days) |

# Support

## Requirements

The application relies on data gathered from gecko and enriched through assure. Both of them needs to be started and properly configured to report valid real time information and to monitor the click to dial service.

## Startup

The application won't allow user to connect until gecko is started.

## Provisioning

When performing assure first time configuration, gecko's address must be provided (the platform) and assure will start retrieving data from gecko's api.

Data will be refreshed every minute.

## Cluster Settings Update

WBA fetches the updated TSS Productivity Tools Cluster settings information for every 5 minutes from the following gecko api.

> (https://TradeSenseIP/gecko/api/provisioning/v1/device-config)

### Errors

| Code | Status | Message | Reason |
|---|---|---|---|
| 500 | Server Error | Internal Server Error | No platform has been added in assure |
| 500 | Server Error | Internal Server Error | Data provisioning still in progress - Please try again later |

# Commands

## Log Level

When asking for support, the user will often be required to provide the logs of the application. The `loglevel` command allows the user to get the current level and to change it.

### Get the current log level

```
{
  "command" : "loglevel",
  "command_ref" : "command reference"
}
```

The response from the server will return the current level.

```
{
  "command": "loglevel",
  "command_ref": "command reference",
  "args": {
    "level": "DEBUG"
  }
}
```

### Change the log level

If the level argument is provided, the current level will be changed.

```
{
  "command" : "loglevel",
  "command_ref" : "command reference",
  "args": {
     "level": "info"
  }
}
```

The response from the server will return the previous and current level.

```
{
  "command": "response",
  "command_ref": "command reference",
  "success": true,
  "data": {
      "previous_level": "DEBUG",
      "current_level": "INFO"
  }
}
```

#### Available levels

The available levels are `warning`, `info`, `debug` and `error`.

#### Errors

| Code | Status | Message | Reason |
|---|---|---|---|
| 400 | Bad Request | The client has made a bad request, which includes requests with malformed JSON, or invalid data values as well as requests in which the URL contains incorrect values | Bad value provided for parameter: level |

## Get Health

This support command provides information about data provisioning status.

#### Get the provisioning status

```
{
  "command" : "get_health",
  "command_ref" : "command reference"
}
```

The response from the server will return completion status for assure and gecko.

```
{
  "command": "response",
  "command_ref": "command reference",
  "success": true,
  "data": {
    "provisioning": {
      "assure": {
        "completed": true,
        "last_refresh": "2021-09-24T12:00:00Z"
      },
      "gecko": {
        "completed": true,
        "recently_updated": true,
        "max_age": 10
      }
    }
  }
}
```

Information about gecko are read from its API health path.

#### Provisioning data items

- `completed` : provisioning completion status
- `last_refresh` : UTC datetime of provisioning completion or last data refresh
- `recently_updated` : true if data has been updated less than max_age ago
- `max_age` : maximum time frame between updates (in minutes)

**Errors**

| Code | Status | Message | Reason |
|---|---|---|---|
| 500 | Server Error | Internal Server Error | Empty response from gecko |
| 500 | Server Error | Internal Server Error | Unable to extract maxAge from gecko's response |

# Document History

| Issue | Version | Date | Description |
|---|---|---|---|
| 1 | 10.0.0 | 10/01/2023 | First issue of Tradesense WBA document 10.0.0(based upon 9.5.18). |
| 2 | 10.0.1 | 10/02/2023 | Introduced Merged state in Get Calls, Get Events and Calls notifications. Handled Session Expired state and ERROR state for calls notifications. |
| 3 | 10.0.2 | 05/04/2023 | Handled Intercom call events & states in call notifications, get calls and get events. Introduced unsubcribe to notifications of category calls and presences. |
| 4 | 10.0.4 | 18/07/2023 | Introduced get_lines, get_tpos, subscribe to alerts notifications, Login and Logout turret for service_ctd command. Introduced Optional argument field 'get_lines_info' for get_users command and added Recording Server information to get_users,get_turrets,get_tpos and presences notification responses. Added linked_cdr_id in get_calls response. |
| 5 | 10.0.5 | 24/08/2023 | Introduced get_health_api_report, upgraded get_users response to be in batches. |
| 6 | 10.0.6 | 02/10/2023 | Introduced a new task to update TSS Productivity Tools Cluster settings for every 5 minutes, Introduced get_shared_profiles. |
| 7 | 10.0.7 | 15/11/2023 | Added Tradesense node health report in get_health_api_report. Added ddi_slot_extension info in get_calls, get_events, subscribe notifications. |
| 8 | 10.0.8 | 23/01/2024 | Removed line_type in get_users, updated the transfer unattended scenario. |
| 9 | 10.0.12 | 09/07/2024 | Added Get Response limit settings for get_calls and get_events. |

| Issue | Version | Date | Description |
|---|---|---|---|
| 10 | 10.0.14 | 30/10/2024 | Updated Get Calls & Get Events info, Added few notes regarding response limit & response batch size settings, Updated get_health_api command with optional argument for alerts data. Removed batch size for notifications. |